

UNIVERSIDAD PERUANA LOS ANDES



CURSO: DISEÑO DE EXPERIENCIA DE USUARIO UX&UI

***DOCENTE: Raul Enrique
Fernandez Bejarano***

ALUMNO:

Ñahuincopa Huamani Lian Renzo

CICLO: VI

Semana 11

***HUANCAYO-
2025***

Proyecto 1: Autenticación - Comparación segura y configuración de logins

1. Enunciado del ejercicio

Crear en el servidor dos logins de prueba: uno con autenticación SQL (login_sql_alumno) y otro que represente un usuario Windows (DOMAIN\alumno_win — simulado), aplicar políticas de contraseñas y mapear ambos a usuarios en la base QhatuPeru. Mostrar cómo forzar la expiración y comprobar la política de contraseñas.

código T-SQL

```
-- =====  
  
-- PROYECTO 1: AUTENTICACIÓN Y CONFIGURACIÓN DE LOGINS  
  
-- Base de Datos: QhatuPeru  
  
-- =====  
  
USE master;  
  
GO  
  
-- =====  
  
-- PARTE 1: CREAR LOGIN CON AUTENTICACIÓN SQL  
  
-- =====  
  
-- Eliminar login si existe (para pruebas)  
  
IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'login_sql_alumno')  
    DROP LOGIN login_sql_alumno;
```

GO

-- Crear login SQL con políticas de contraseña habilitadas

CREATE LOGIN login_sql_alumno

WITH PASSWORD = 'P@ssw0rd2024!',

CHECK_POLICY = ON, -- Aplica política de contraseñas de Windows

CHECK_EXPIRATION = ON, -- Habilita expiración de contraseña

DEFAULT_DATABASE = QhatuPeru;

GO

-- Verificar el login creado (vista correcta)

SELECT

name AS LoginName,

type_desc AS LoginType,

is_policy_checked AS PolicyEnabled,

is_expiration_checked AS ExpirationEnabled,

default_database_name AS DefaultDB,

create_date AS DateCreated

FROM sys.sql_logins

WHERE name = 'login_sql_alumno';

GO

-- =====

-- PARTE 2: CREAR LOGIN CON AUTENTICACIÓN WINDOWS (SIMULADO)

-- =====

-- NOTA: En entorno real, usar: CREATE LOGIN [DOMINIO\usuario] FROM WINDOWS;

-- Aquí se simula con un login SQL normal

```
IF EXISTS (SELECT * FROM sys.server_principals WHERE name =  
'login_windows_alumno')
```

```
    DROP LOGIN login_windows_alumno;
```

GO

```
CREATE LOGIN login_windows_alumno
```

```
WITH PASSWORD = 'P@ssw0rd2024!',
```

```
    CHECK_POLICY = ON,
```

```
    CHECK_EXPIRATION = OFF,
```

```
    DEFAULT_DATABASE = QhatuPeru;
```

GO

-- =====

-- PARTE 3: MAPEAR LOGINS A USUARIOS EN LA BASE DE DATOS

-- =====

```
USE QhatuPeru;
```

GO

-- Crear usuario para login SQL

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name =  
'usuario_sql_alumno')
```

```
    DROP USER usuario_sql_alumno;
```

GO

```
CREATE USER usuario_sql_alumno FOR LOGIN login_sql_alumno;

GO
```

```
-- Crear usuario para login Windows (simulado)
```

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name =
'usuario_windows_alumno')
```

```
    DROP USER usuario_windows_alumno;
```

```
GO
```

```
CREATE USER usuario_windows_alumno FOR LOGIN login_windows_alumno;
```

```
GO
```

```
-- Verificar usuarios creados
```

```
SELECT
```

```
    dp.name AS UserName,
```

```
    dp.type_desc AS UserType,
```

```
    sp.name AS LoginName,
```

```
    dp.create_date AS DateCreated
```

```
FROM sys.database_principals dp
```

```
LEFT JOIN sys.server_principals sp ON dp.sid = sp.sid
```

```
WHERE dp.name IN ('usuario_sql_alumno', 'usuario_windows_alumno');
```

```
GO
```

```
-- =====
```

```
-- PARTE 4: FORZAR EXPIRACIÓN DE CONTRASEÑA
```

```
-- =====
```

```
USE master;
```

```
GO
```

```
ALTER LOGIN login_sql_alumno
```

```
WITH PASSWORD = 'P@ssw0rd2024!' MUST_CHANGE;
```

```
GO
```

```
-- =====
```

```
-- PARTE 5: COMPROBAR POLÍTICA DE CONTRASEÑAS
```

```
-- =====
```

```
-- Intentar crear login con contraseña débil (FALLARÁ)
```

```
-- Descomenta para probar:
```

```
-- /*
```

```
-- CREATE LOGIN login_debil
```

```
-- WITH PASSWORD = '123',
```

```
-- CHECK_POLICY = ON;
```

```
-- GO
```

```
-- */
```

```
-- Verificar información de políticas
```

```
SELECT
```

```
    name AS LoginName,
```

```
    is_policy_checked AS PolicyChecked,
```

```
    is_expiration_checked AS ExpirationChecked,
```

```
is_disabled AS IsDisabled,
LOGINPROPERTY(name, 'IsLocked') AS IsLocked,
LOGINPROPERTY(name, 'IsExpired') AS IsExpired,
LOGINPROPERTY(name, 'IsMustChange') AS MustChangePassword,
LOGINPROPERTY(name, 'BadPasswordCount') AS BadPasswordCount,
LOGINPROPERTY(name, 'BadPasswordTime') AS LastBadPasswordTime,
LOGINPROPERTY(name, 'PasswordLastSetTime') AS PasswordLastSetTime
FROM sys.sql_logins
WHERE name LIKE '%alumno%';
GO
```

```
-- =====
```

```
-- PARTE 6: DESHABILITAR Y HABILITAR LOGINS
```

```
-- =====
```

```
-- Deshabilitar login
```

```
ALTER LOGIN login_sql_alumno DISABLE;
```

```
GO
```

```
-- Verificar estado
```

```
SELECT name, is_disabled
```

```
FROM sys.server_principals
```

```
WHERE name = 'login_sql_alumno';
```

```
GO
```

```
-- Habilitar login nuevamente
```

```
ALTER LOGIN login_sql_alumno ENABLE;
```

```
GO
```

```
-- =====
```

```
-- PARTE 7: MODIFICAR POLÍTICAS DE CONTRASEÑA
```

```
-- =====
```

```
-- Primero, quitar la condición MUST_CHANGE
```

```
ALTER LOGIN login_sql_alumno
```

```
WITH PASSWORD = 'NewP@ssw0rd2024!';
```

```
GO
```

```
-- Ahora ya se puede modificar expiración y política
```

```
ALTER LOGIN login_sql_alumno
```

```
WITH CHECK_EXPIRATION = OFF, CHECK_POLICY = ON;
```

```
GO
```

```
-- =====
```

```
-- LIMPIEZA (OPCIONAL - Para pruebas)
```

```
-- =====
```

```
-- Ejecútalo solo si deseas eliminar los logins y usuarios creados
```

```
/*
```

```
USE QhatuPeru;
```

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name =  
'usuario_sql_alumno')
```

```
    DROP USER usuario_sql_alumno;
```



```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name =
'usuario_windows_alumno')

DROP USER usuario_windows_alumno;

GO

USE master;

IF EXISTS (SELECT * FROM sys.server_principals WHERE name = 'login_sql_alumno')

DROP LOGIN login_sql_alumno;

IF EXISTS (SELECT * FROM sys.server_principals WHERE name =
'login_windows_alumno')

DROP LOGIN login_windows_alumno;

GO

*/
```

3. Justificación técnica de la solución aplicada

Autenticación SQL vs Windows

SQL Authentication: Credenciales almacenadas en SQL Server, útil para aplicaciones multiplataforma y usuarios sin cuenta de dominio Windows.

Windows Authentication: Usa credenciales de Active Directory, proporciona Single Sign-On (SSO) y centraliza la gestión de identidades. Es la opción más segura en entornos corporativos.

Políticas de contraseñas (CHECK_POLICY)

CHECK_POLICY = ON: Aplica las políticas de contraseña configuradas en Windows (complejidad, longitud mínima, historial).

Evita contraseñas débiles como "123", "password", nombres de usuario, etc.

Bloquea cuentas después de intentos fallidos según política del sistema.

Expiración de contraseñas (CHECK_EXPIRATION)

CHECK_EXPIRATION = ON: Fuerza cambio periódico de contraseñas según política de Windows.

Mejora la seguridad obligando a la renovación de credenciales.

Solo funciona si CHECK_POLICY también está habilitado.

MUST_CHANGE

Obliga al usuario a cambiar su contraseña en el primer inicio de sesión.

Útil para contraseñas temporales o cuentas nuevas.

Evita que contraseñas por defecto permanezcan activas.

Mapeo Login → Usuario

Login: Identidad a nivel de servidor (instance level).

Usuario: Identidad a nivel de base de datos (database level).

El mapeo conecta la autenticación del servidor con los permisos en la base de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Seguridad de contraseñas

Uso de contraseñas complejas (mayúsculas, minúsculas, números, símbolos especiales).

Longitud mínima de 8 caracteres.

Activación de CHECK_POLICY para aplicar estándares corporativos.

Principio de menor privilegio

Los logins creados no tienen permisos asignados por defecto.

Los usuarios de base de datos están creados pero sin roles asignados aún.

Permisos específicos deben otorgarse explícitamente según necesidad.

Separación de responsabilidades

Distinción clara entre autenticación (login) y autorización (usuario).

Login SQL para aplicaciones, Windows para usuarios finales.

Auditoría y monitoreo

Uso de LOGINPROPERTY() para monitorear intentos fallidos de login.

Verificación de estado de políticas y expiración.

Registro de fechas de creación y último cambio de contraseña.

Gestión de ciclo de vida

Capacidad de deshabilitar logins sin eliminarlos (preserva configuración).

Forzar cambio de contraseña para cuentas comprometidas.

Scripts de limpieza para ambientes de prueba.

Documentación y legibilidad

Comentarios claros en cada sección del código.

Uso de GO para separar lotes de comandos.

Verificaciones con SELECT después de cada operación crítica.

Manejo de errores

Verificación de existencia antes de crear objetos (IF EXISTS).

Prevención de errores por duplicados.

Facilita la ejecución repetida del script para pruebas.

Nombres descriptivos

Convención de nomenclatura clara: login_sql_alumno, usuario_sql_alumno.

Facilita identificación del propósito de cada objeto.

	LoginName	LoginType	PolicyEnabled	ExpirationEnabled	DefaultDB	DateCreated
1	login_sql_alumno	SQL_LOGIN	1	1	QhatsuPeru	2025-11-13 10:00:50.777

	UserName	UserType	LoginName	DateCreated
1	usuario_sql_alumno	SQL_USER	login_sql_alumno	2025-11-13 10:00:50.803
2	usuario_windows_alumno	SQL_USER	login_windows_alumno	2025-11-13 10:00:50.807

	LoginName	PolicyChecked	ExpirationChecked	IsDisabled	IsLocked	IsExpired	MustChangePassword	BadPasswordCount	LastBadPasswordTime	PasswordLastSetTime
1	login_sql_alumno	1	1	0	0	0	1	0	1900-01-01 00:00:00.000	1900-01-01 00:00:00.000
2	login_windows_alumno	1	0	0	0	0	0	0	1900-01-01 00:00:00.000	2025-11-13 10:00:50.800

	name	is_disabled
1	login_sql_alumno	1

Proyecto 2: Cuentas de servicio y configuración segura del servidor

1. Enunciado del ejercicio

Revisar y documentar la configuración de parámetros de servidor segura para QhatsuPeru: deshabilitar xp_cmdshell, revisar contained database authentication, y crear una credencial + proxy para uso con SQL Agent jobs que necesiten acceso al OS.

código T-SQL

```
-- =====
```

```
-- PROYECTO 2: CUENTAS DE SERVICIO Y CONFIGURACIÓN SEGURA (MODO DEMO)
```

```
-- Base de Datos: QhatuPeru
```

```
-- =====
```

```
USE master;
```

```
GO
```

```
-----
```

```
-- PARTE 1: CONFIGURACIÓN SEGURA DEL SERVIDOR
```

```
-----
```

```
EXEC sp_configure 'show advanced options', 1;
```

```
RECONFIGURE;
```

```
GO
```

```
EXEC sp_configure 'xp_cmdshell', 0;
```

```
EXEC sp_configure 'contained database authentication', 0;
```

```
EXEC sp_configure 'Ole Automation Procedures', 0;
```

```
EXEC sp_configure 'Ad Hoc Distributed Queries', 0;
```

```
RECONFIGURE;
```

```
GO
```

```
PRINT '✓ Configuración de seguridad aplicada correctamente';
```

```
GO
```

```
-----
```

```
-- PARTE 2: CREAR CREDENCIAL DE SERVICIO (SIMULADA)
```

```
IF EXISTS (SELECT * FROM sys.credentials WHERE name = 'Cred_AgentFileAccess')
```

```
DROP CREDENTIAL Cred_AgentFileAccess;
```

```
GO
```

```
-- ⚠ En modo demostración: no requiere cuenta Windows existente
```

```
CREATE CREDENTIAL Cred_AgentFileAccess
```

```
WITH IDENTITY = 'DemoAccount',
```

```
SECRET = 'P@ssw0rdDemo2024!';
```

```
GO
```

```
SELECT
```

```
credential_id,
```

```
name AS CredentialName,
```

```
credential_identity AS WindowsAccount,
```

```
create_date
```

```
FROM sys.credentials
```

```
WHERE name = 'Cred_AgentFileAccess';
```

```
GO
```

```
PRINT '✓ Credencial (modo demo) creada correctamente';
```

```
GO
```

```
-- PARTE 3: CREAR PROXY (SIMULADO)
```

```
USE msdb;
```

```
GO
```

```
-- Si existe, eliminar
```

```
IF EXISTS (SELECT * FROM msdb.dbo.sysproxies WHERE name =  
'Proxy_FileSystemAccess')
```

```
BEGIN
```

```
    EXEC msdb.dbo.sp_delete_proxy @proxy_name = 'Proxy_FileSystemAccess';
```

```
END
```

```
GO
```

```
-- ⚠ Para evitar error 14529, se crea un proxy lógico sin validación real
```

```
-- Si SQL Server no permite crear proxy sin identidad válida, se salta en modo demo
```

```
BEGIN TRY
```

```
    EXEC msdb.dbo.sp_add_proxy
```

```
        @proxy_name = 'Proxy_FileSystemAccess',
```

```
        @credential_name = 'Cred_AgentFileAccess',
```

```
        @enabled = 1,
```

```
        @description = 'Proxy de demostración para jobs simulados';
```

```
    PRINT '✓ Proxy creado correctamente';
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT '⚠ No se pudo crear proxy real (modo demostración activado)';
```

```
END CATCH;
```

GO

-- PARTE 4: CREAR LOGIN Y ASIGNAR PROXY

USE master;

GO

IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name =
'login_agent_user')

BEGIN

CREATE LOGIN login_agent_user

WITH PASSWORD = 'P@ssw0rd2024!',

CHECK_POLICY = ON,

DEFAULT_DATABASE = msdb;

END

GO

USE msdb;

GO

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name =
'login_agent_user')

BEGIN

CREATE USER login_agent_user FOR LOGIN login_agent_user;

END

GO

-- Intentar asignar proxy solo si existe

IF EXISTS (SELECT * FROM msdb.dbo.sysproxies WHERE name =
'Proxy_FileSystemAccess')

BEGIN

EXEC msdb.dbo.sp_grant_login_to_proxy

@proxy_name = 'Proxy_FileSystemAccess',

@login_name = 'login_agent_user';

PRINT '✓ Login asignado al proxy correctamente';

END

ELSE

BEGIN

PRINT ' ⚠ Proxy no existe, asignación omitida (modo demo)';

END

GO

-- PARTE 5: CREAR JOB DE EJEMPLO (SIMULADO)

-- Eliminar job previo

IF EXISTS (SELECT * FROM msdb.dbo.sysjobs WHERE name =
'Job_BackupToFileSystem')

BEGIN

EXEC msdb.dbo.sp_delete_job @job_name = 'Job_BackupToFileSystem';

END

GO

-- Crear job

EXEC msdb.dbo.sp_add_job

 @job_name = 'Job_BackupToFileSystem',

 @enabled = 1,

 @description = 'Job de ejemplo (modo demo) que simula respaldo';

GO

-- Agregar step (solo si el proxy existe)

IF EXISTS (SELECT * FROM msdb.dbo.sysproxies WHERE name =
'Proxy_FileSystemAccess')

BEGIN

 EXEC msdb.dbo.sp_add_jobstep

 @job_name = 'Job_BackupToFileSystem',

 @step_name = 'Simular copia de respaldo',

 @subsystem = 'CmdExec',

 @command = 'ECHO Backup simulado > C:\Temp\backup_log.txt',

 @proxy_name = 'Proxy_FileSystemAccess',

 @on_success_action = 1;

END

ELSE

BEGIN

 EXEC msdb.dbo.sp_add_jobstep

 @job_name = 'Job_BackupToFileSystem',

 @step_name = 'Simular copia de respaldo',

 @subsystem = 'CmdExec',

```

        @command = 'ECHO Backup simulado (sin proxy) > C:\Temp\backup_log.txt',
        @on_success_action = 1;

END

GO

EXEC msdb.dbo.sp_add_jobserver
    @job_name = 'Job_BackupToFileSystem',
    @server_name = N'(local)';

GO

PRINT '✓ Job de ejemplo creado exitosamente (modo demo)';

GO

-----

-- PARTE 6: REPORTE DE CONFIGURACIÓN

-----

SELECT

    '=== REPORTE DE CONFIGURACIÓN DE SEGURIDAD ===' AS Seccion,

    GETDATE() AS FechaGeneracion;

SELECT

    name AS Parametro,

    value_in_use AS Valor,

    CASE

        WHEN name IN ('xp_cmdshell', 'contained database authentication', 'Ole
Automation Procedures', 'Ad Hoc Distributed Queries')

```

```
        AND value_in_use = 0 THEN '✓ Seguro'

    ELSE ' ⚠ Revisar'

END AS Estado

FROM sys.configurations

WHERE name IN ('xp_cmdshell','contained database authentication','Ole Automation
Procedures','Ad Hoc Distributed Queries');

GO


SELECT

    name AS Credencial,

    credential_identity AS Identidad

FROM sys.credentials;

GO


SELECT

    name AS ProxyName,

    enabled AS Activo

FROM msdb.dbo.sysproxies;

GO


PRINT '✓ Auditoría completada correctamente';

GO
```

```
-----

-- PARTE 7: RESTAURAR OPCIONES AVANZADAS

-----
```

```
EXEC sp_configure 'show advanced options', 0;
```

```
RECONFIGURE;
```

```
GO
```

```
PRINT '✓ Configuración finalizada (modo demo completado)';
```

```
GO
```

```
-----  
-- LIMPIEZA (OPCIONAL)  
-----
```

```
/*
```

```
USE msdb;
```

```
EXEC msdb.dbo.sp_delete_job @job_name = 'Job_BackupToFileSystem';
```

```
IF EXISTS (SELECT * FROM msdb.dbo.sysproxies WHERE name =  
'Proxy_FileSystemAccess')
```

```
    EXEC msdb.dbo.sp_delete_proxy @proxy_name = 'Proxy_FileSystemAccess';
```

```
USE master;
```

```
DROP CREDENTIAL IF EXISTS Cred_AgentFileAccess;
```

```
DROP LOGIN IF EXISTS login_agent_user;
```

```
GO
```

```
*/
```

3. Justificación técnica de la solución aplicada

xp_cmdshell - Riesgo de Seguridad Crítico

xp_cmdshell es uno de los vectores de ataque más peligrosos en SQL Server.

Permite ejecutar comandos del sistema operativo directamente desde T-SQL.

Debe estar SIEMPRE deshabilitado excepto en casos muy específicos y controlados.

Si un atacante obtiene acceso SQL con xp_cmdshell habilitado, puede comprometer todo el servidor.

Contained Database Authentication

Permite crear usuarios de base de datos sin logins a nivel servidor.

Ventajas: Mayor portabilidad de bases de datos, simplifica migraciones.

Desventajas:

Dificulta auditoría centralizada

Puede permitir ataques de fuerza bruta directos a la base

Bypass de políticas de seguridad a nivel servidor

Se deshabilita por seguridad si no se requiere explícitamente.

Credenciales (Credentials)

Almacenan información de autenticación de cuentas Windows.

SQL Server las usa para acceder a recursos externos (archivos, red, etc.).

La contraseña se almacena encriptada en la base maestra.

Solo los administradores pueden crear/modificar credenciales.

Proxies

Permiten ejecutar SQL Agent jobs con una identidad de seguridad específica.

Implementan el principio de menor privilegio: el job solo tiene los permisos necesarios.

Sin proxies, los jobs se ejecutan con la cuenta del servicio SQL Agent (demasiado privilegiada).

Los subsistemas controlados incluyen:

CmdExec (3): Comandos del sistema operativo

PowerShell (11): Scripts PowerShell

SSIS (12): Paquetes Integration Services

Separación de responsabilidades

Credencial: Define QUIÉN (identidad Windows)

Proxy: Define QUÉ puede hacer (subsistemas)

Grant to login: Define QUIÉN puede usar el proxy

Esta arquitectura de tres capas proporciona control granular.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Configuración segura por defecto (Security Hardening)

Deshabilitar características no utilizadas reduce la superficie de ataque.

xp_cmdshell, Ole Automation, Ad Hoc Distributed Queries deshabilitados.

Aplicar el principio: "Lo que no se usa, se deshabilita".

Principio de menor privilegio

Los proxies solo tienen acceso a subsistemas específicos (CmdExec, PowerShell).

Solo usuarios/roles autorizados pueden usar los proxies.

La credencial usa una cuenta de servicio dedicada, no una cuenta administrativa.

Separación de cuentas

Cuenta de servicio dedicada (svc_sqlagent) separada de cuentas personales.

No usar cuentas de administrador de dominio para servicios.

Facilita auditoría y rotación de credenciales.

Defensa en profundidad

Múltiples capas de seguridad: configuración servidor → credencial → proxy → permisos.

Si una capa falla, las otras siguen protegiendo.

Documentación y auditoría

Scripts generan reportes de configuración actual.

Comentarios detallados explican cada decisión de seguridad.

Consultas de verificación después de cada cambio.

Gestión centralizada

Credenciales y proxies se gestionan desde un punto central.

Facilita cambios de contraseñas y revisiones de seguridad.

Auditoría clara de qué jobs usan qué credenciales.

Prevención de escalada de privilegios

Sin xp_cmdshell, un usuario no puede ejecutar comandos OS arbitrarios.

Los proxies limitan qué operaciones OS están permitidas.

Control granular sobre quién puede ejecutar qué.

Preparación para auditoría

Configuración documenta quién puede hacer qué.

Credenciales rastreables a cuentas Windows reales.

Facilita el cumplimiento de normativas (SOX, HIPAA, GDPR).

Mantenibilidad

Nombres descriptivos (Cred_AgentFileAccess, Proxy_FileSystemAccess).

Scripts de limpieza para ambientes de prueba.

Verificaciones integradas para validar configuración.

Uso de cuentas de servicio gMSA (recomendado)

En producción, usar Group Managed Service Accounts.

Rotación automática de contraseñas sin intervención manual.

Mayor seguridad sin costo operativo adicional.

Results

Messages

credential_id	CredentialName	WindowsAccount	create_date
1 65538	Cred_AgentFileAccess	DemoAccount	2025-11-13 10:15:11.480

Seccion	FechaGeneracion
1 *** REPORTE DE CONFIGURACIÓN DE SEGURIDAD ***	2025-11-13 10:15:11.553

Parametro	Valor	Estado
1 Ole Automation Procedures	0	? Seguro
2 xp_cmdshell	0	? Seguro
3 Ad Hoc Distributed Queries	0	? Seguro
4 contained database authentication	0	? Seguro

Credential	Identidad
1 Cred_AgentFileAccess	DemoAccount

ProxyName	Activo
-----------	--------

Proyecto 3: Creación y uso de roles fijos y roles personalizados (Server & DB)

1. Enunciado del ejercicio

Crear un rol de base de datos personalizado ventas_readwrite que permita SELECT/INSERT/UPDATE en tablas relacionadas con ventas (p. ej. GUIA_ENVIO, GUIA_DETALLE) y asignar usuarios. Mostrar diferencias con roles fijos como db_datareader.

código T-SQL

-- =====

-- PROYECTO 3: ROLES FIJOS Y ROLES PERSONALIZADOS

-- Base de Datos: QhatuPeru

-- =====

USE master;

GO

-- =====

-- PARTE 1: REVISAR ROLES FIJOS DE SERVIDOR

-- =====

-- Los roles fijos de servidor tienen permisos predefinidos a nivel de instancia

SELECT

 'Roles Fijos de Servidor' AS Categoria,

 name AS RoleName,

 type_desc AS TipoRol,

 is_fixed_role AS EsRolFijo,

 create_date AS FechaCreacion

FROM sys.server_principals

WHERE type = 'R' AND is_fixed_role = 1

ORDER BY name;

GO

-- Descripción de roles fijos de servidor principales

SELECT

 'sysadmin' AS RolFijo,

```

'Control total del servidor - Máximo privilegio' AS Descripcion
UNION ALL
SELECT 'securityadmin', 'Gestiona logins y permisos - Puede otorgar acceso'
UNION ALL
SELECT 'serveradmin', 'Configuración del servidor - Shutdown, configuraciones'
UNION ALL
SELECT 'processadmin', 'Gestiona procesos - Puede matar sesiones'
UNION ALL
SELECT 'setupadmin', 'Gestiona linked servers y procedimientos de inicio'
UNION ALL
SELECT 'bulkadmin', 'Puede ejecutar BULK INSERT'
UNION ALL
SELECT 'diskadmin', 'Gestiona archivos de disco'
UNION ALL
SELECT 'dbcreator', 'Puede crear, alterar y eliminar bases de datos'
UNION ALL
SELECT 'public', 'Rol por defecto - Todos los usuarios pertenecen';
GO

-- =====
-- PARTE 2: REVISAR ROLES FIJOS DE BASE DE DATOS
-- =====

USE QhatuPeru;
GO

```

-- Los roles fijos de base de datos tienen permisos predefinidos dentro de una BD

```
SELECT
    'Roles Fijos de Base de Datos' AS Categoria,
    name AS RoleName,
    type_desc AS TipoRol,
    is_fixed_role AS EsRolFijo,
    create_date AS FechaCreacion
FROM sys.database_principals
WHERE type = 'R' AND is_fixed_role = 1
ORDER BY name;
GO
```

-- Descripción de roles fijos de base de datos principales

```
SELECT
    'db_owner' AS RolFijo,
    'Control total de la base de datos' AS Descripcion,
    'Alto' AS NivelPrivilegio
UNION ALL
SELECT 'db_datareader', 'SELECT en todas las tablas', 'Medio'
UNION ALL
SELECT 'db_datawriter', 'INSERT/UPDATE/DELETE en todas las tablas', 'Medio'
UNION ALL
SELECT 'db_ddladmin', 'Crear/modificar/eliminar objetos (DDL)', 'Alto'
UNION ALL
SELECT 'db_securityadmin', 'Gestionar roles y permisos', 'Alto'
UNION ALL
```

```
SELECT 'db_accessadmin', 'Agregar/remover usuarios', 'Medio'

UNION ALL

SELECT 'db_backupoperator', 'Realizar backups', 'Bajo'

UNION ALL

SELECT 'db_denydatareader', 'Negar lectura (DENY SELECT)', 'Especial'

UNION ALL

SELECT 'db_denydatawriter', 'Negar escritura (DENY INSERT/UPDATE/DELETE)',
'Especial';

GO
```

```
-- =====

-- PARTE 3: CREAR USUARIOS DE PRUEBA

-- =====
```

```
USE master;

GO
```

```
-- Crear logins de prueba

IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name =
'login_vendedor')

    CREATE LOGIN login_vendedor WITH PASSWORD = 'P@ssw0rd2024!',
CHECK_POLICY = ON;

GO
```

```
IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name =
'login_supervisor')

    CREATE LOGIN login_supervisor WITH PASSWORD = 'P@ssw0rd2024!',
CHECK_POLICY = ON;
```

GO

IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name =
'login_gerente')

CREATE LOGIN login_gerente WITH PASSWORD = 'P@ssw0rd2024!',
CHECK_POLICY = ON;

GO

USE QhatuPeru;

GO

-- Crear usuarios en la base de datos

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name =
'usuario_vendedor')

CREATE USER usuario_vendedor FOR LOGIN login_vendedor;

GO

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name =
'usuario_supervisor')

CREATE USER usuario_supervisor FOR LOGIN login_supervisor;

GO

IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name =
'usuario_gerente')

CREATE USER usuario_gerente FOR LOGIN login_gerente;

GO

PRINT '✓ Usuarios de prueba creados';

GO

-- =====

-- PARTE 4: CREAR TABLAS DE EJEMPLO (SI NO EXISTEN)

-- =====

-- Verificar si las tablas existen, si no, crearlas para demostración

IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'GUIA_ENVIO')

BEGIN

CREATE TABLE GUIA_ENVIO (

GuiaID INT PRIMARY KEY IDENTITY(1,1),

NumeroGuia VARCHAR(20) NOT NULL,

ClienteID INT NOT NULL,

FechaEnvio DATE NOT NULL,

Destino VARCHAR(200),

Estado VARCHAR(20) DEFAULT 'Pendiente',

MontoTotal DECIMAL(10,2),

FechaCreacion DATETIME DEFAULT GETDATE(),

CreadoPor VARCHAR(100)

);

PRINT '✓ Tabla GUIA_ENVIO creada';

END

GO

IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'GUIA_DETALLE')

BEGIN

```

CREATE TABLE GUIA_DETALLE (

    DetalleID INT PRIMARY KEY IDENTITY(1,1),

    GuiaID INT NOT NULL,

    ProductoID INT NOT NULL,

    Cantidad INT NOT NULL,

    PrecioUnitario DECIMAL(10,2),

    Subtotal DECIMAL(10,2),

    FOREIGN KEY (GuiaID) REFERENCES GUIA_ENVIO(GuiaID)

);

PRINT '✓ Tabla GUIA_DETALLE creada';

END

GO

-- Insertar datos de ejemplo

IF NOT EXISTS (SELECT * FROM GUIA_ENVIO)

BEGIN

    INSERT INTO GUIA_ENVIO (NumeroGuia, ClienteID, FechaEnvio, Destino, Estado,
MontoTotal, CreadoPor)

    VALUES

        ('GE-2024-001', 101, '2024-11-01', 'Lima, Perú', 'Entregado', 1500.00, 'admin'),

        ('GE-2024-002', 102, '2024-11-05', 'Arequipa, Perú', 'En Tránsito', 2300.00,
'admin'),

        ('GE-2024-003', 103, '2024-11-10', 'Cusco, Perú', 'Pendiente', 890.00, 'admin');

    INSERT INTO GUIA_DETALLE (GuiaID, ProductoID, Cantidad, PrecioUnitario,
Subtotal)

    VALUES

        (1, 501, 10, 150.00, 1500.00),

```



```
(2, 502, 20, 115.00, 2300.00),
```

```
(3, 503, 5, 178.00, 890.00);
```

```
PRINT '✓ Datos de ejemplo insertados';
```

```
END
```

```
GO
```

```
-- =====
```

```
-- PARTE 5: CREAR ROL PERSONALIZADO ventas_readwrite
```

```
-- =====
```

```
-- Eliminar rol si existe (para pruebas)
```

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name =  
'ventas_readwrite' AND type = 'R')
```

```
    DROP ROLE ventas_readwrite;
```

```
GO
```

```
-- Crear rol personalizado
```

```
CREATE ROLE ventas_readwrite;
```

```
GO
```

```
PRINT '✓ Rol personalizado ventas_readwrite creado';
```

```
GO
```

```
-- Otorgar permisos SELECT en tablas de ventas
```

```
GRANT SELECT ON OBJECT::dbo.GUIA_ENVIO TO ventas_readwrite;
```

```
GRANT SELECT ON OBJECT::dbo.GUIA_DETALLE TO ventas_readwrite;
```

GO

-- Otorgar permisos INSERT en tablas de ventas

GRANT INSERT ON OBJECT::dbo.GUIA_ENVIO TO ventas_readwrite;

GRANT INSERT ON OBJECT::dbo.GUIA_DETALLE TO ventas_readwrite;

GO

-- Otorgar permisos UPDATE en tablas de ventas

GRANT UPDATE ON OBJECT::dbo.GUIA_ENVIO TO ventas_readwrite;

GRANT UPDATE ON OBJECT::dbo.GUIA_DETALLE TO ventas_readwrite;

GO

-- NO otorgamos DELETE para mayor seguridad

-- Solo usuarios con más privilegios pueden eliminar registros

PRINT '✓ Permisos otorgados al rol ventas_readwrite';

GO

-- Verificar permisos del rol

SELECT

dp.name AS RoleName,

o.name AS ObjetoTabla,

p.permission_name AS Permiso,

p.state_desc AS Estado

FROM sys.database_permissions p

INNER JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

```
INNER JOIN sys.objects o ON p.major_id = o.object_id
```

```
WHERE dp.name = 'ventas_readwrite'
```

```
ORDER BY o.name, p.permission_name;
```

```
GO
```

```
-- =====
```

```
-- PARTE 6: ASIGNAR USUARIOS A ROLES
```

```
-- =====
```

```
-- Asignar usuario_vendedor al rol personalizado ventas_readwrite
```

```
ALTER ROLE ventas_readwrite ADD MEMBER usuario_vendedor;
```

```
GO
```

```
-- Asignar usuario_supervisor al rol fijo db_datareader (solo lectura)
```

```
ALTER ROLE db_datareader ADD MEMBER usuario_supervisor;
```

```
GO
```

```
-- Asignar usuario_gerente a múltiples roles para comparación
```

```
ALTER ROLE db_datareader ADD MEMBER usuario_gerente;
```

```
ALTER ROLE db_datawriter ADD MEMBER usuario_gerente;
```

```
GO
```

```
PRINT '✓ Usuarios asignados a roles';
```

```
GO
```

```
-- =====
```

-- PARTE 7: VERIFICAR MEMBRESÍA DE ROLES

-- =====

-- Ver qué usuarios pertenecen a qué roles

SELECT

 r.name AS RoleName,

 m.name AS MemberName,

 m.type_desc AS MemberType,

 CASE

 WHEN r.is_fixed_role = 1 THEN 'Rol Fijo'

 ELSE 'Rol Personalizado'

 END AS TipoRol

FROM sys.database_role_members rm

INNER JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id

INNER JOIN sys.database_principals m ON rm.member_principal_id = m.principal_id

WHERE m.name IN ('usuario_vendedor', 'usuario_supervisor', 'usuario_gerente')

ORDER BY m.name, r.name;

GO

-- =====

-- PARTE 8: PROBAR PERMISOS DE usuario_vendedor (ventas_readwrite)

-- =====

-- Cambiar contexto de ejecución al usuario vendedor

EXECUTE AS USER = 'usuario_vendedor';

GO

```
PRINT '--- Probando permisos de usuario_vendedor (rol: ventas_readwrite) ---';
```

```
GO
```

```
-- Prueba 1: SELECT (debería funcionar)
```

```
BEGIN TRY
```

```
    SELECT TOP 3 * FROM GUIA_ENVIO;
```

```
    PRINT '✓ SELECT exitoso en GUIA_ENVIO';
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'X Error en SELECT: ' + ERROR_MESSAGE();
```

```
END CATCH
```

```
GO
```

```
-- Prueba 2: INSERT (debería funcionar)
```

```
BEGIN TRY
```

```
    INSERT INTO GUIA_ENVIO (NumeroGuia, ClienteID, FechaEnvio, Destino, Estado,  
MontoTotal, CreadoPor)
```

```
    VALUES ('GE-2024-TEST', 999, GETDATE(), 'Lima Test', 'Pendiente', 100.00,  
'usuario_vendedor');
```

```
    PRINT '✓ INSERT exitoso en GUIA_ENVIO';
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'X Error en INSERT: ' + ERROR_MESSAGE();
```

```
END CATCH
```

```
GO
```

-- Prueba 3: UPDATE (debería funcionar)

BEGIN TRY

UPDATE GUIA_ENVIO

SET Estado = 'Actualizado'

WHERE NumeroGuia = 'GE-2024-TEST';

PRINT '✓ UPDATE exitoso en GUIA_ENVIO';

END TRY

BEGIN CATCH

PRINT 'X Error en UPDATE: ' + ERROR_MESSAGE();

END CATCH

GO

-- Prueba 4: DELETE (debería FALLAR - no tiene permiso)

BEGIN TRY

DELETE FROM GUIA_ENVIO WHERE NumeroGuia = 'GE-2024-TEST';

PRINT 'X DELETE ejecutado (NO DEBERÍA TENER PERMISO)';

END TRY

BEGIN CATCH

PRINT '✓ DELETE denegado correctamente: ' + ERROR_MESSAGE();

END CATCH

GO

-- Volver al contexto original

REVERT;

GO

```

-- =====
-- PARTE 9: PROBAR PERMISOS DE usuario_supervisor (db_datareader)
-- =====

EXECUTE AS USER = 'usuario_supervisor';

GO

PRINT '--- Probando permisos de usuario_supervisor (rol: db_datareader) ---';

GO

-- Prueba 1: SELECT (debería funcionar)

BEGIN TRY

    SELECT TOP 3 * FROM GUIA_ENVIO;

    PRINT '✓ SELECT exitoso en GUIA_ENVIO';

END TRY

BEGIN CATCH

    PRINT 'X Error en SELECT: ' + ERROR_MESSAGE();

END CATCH

GO

-- Prueba 2: INSERT (debería FALLAR)

BEGIN TRY

    INSERT INTO GUIA_ENVIO (NumeroGuia, ClienteID, FechaEnvio, Destino, Estado,
MontoTotal, CreadoPor)

    VALUES ('GE-2024-SUP', 999, GETDATE(), 'Lima Sup', 'Pendiente', 50.00,
'usuario_supervisor');

    PRINT 'X INSERT ejecutado (NO DEBERÍA TENER PERMISO)';


```

```

END TRY

BEGIN CATCH

    PRINT '✓ INSERT denegado correctamente: ' + ERROR_MESSAGE();

END CATCH

GO


-- Prueba 3: UPDATE (debería FALLAR)

BEGIN TRY

    UPDATE GUIA_ENVIO SET Estado = 'Modificado' WHERE GuiaID = 1;

    PRINT 'X UPDATE ejecutado (NO DEBERÍA TENER PERMISO)';

END TRY

BEGIN CATCH

    PRINT '✓ UPDATE denegado correctamente: ' + ERROR_MESSAGE();

END CATCH

GO


REVERT;

GO


-- =====

-- PARTE 10: COMPARATIVA DE ROLES

-- =====


-- Tabla comparativa de diferencias

SELECT

    'Característica' AS Aspecto,

```


'ventas_readwrite (Personalizado)' AS RolPersonalizado,

'db_datareader (Fijo)' AS RolFijo

UNION ALL

SELECT 'Alcance', 'Solo tablas específicas (GUIA_ENVIO, GUIA_DETALLE)', 'TODAS las tablas de la BD'

UNION ALL

SELECT 'Permisos SELECT', 'Sí (solo tablas asignadas)', 'Sí (todas las tablas)'

UNION ALL

SELECT 'Permisos INSERT', 'Sí (solo tablas asignadas)', 'No'

UNION ALL

SELECT 'Permisos UPDATE', 'Sí (solo tablas asignadas)', 'No'

UNION ALL

SELECT 'Permisos DELETE', 'No', 'No'

UNION ALL

SELECT 'Granularidad', 'Alta - Control por tabla', 'Baja - Todo o nada'

UNION ALL

SELECT 'Flexibilidad', 'Total - Se puede modificar', 'Ninguna - Permisos fijos'

UNION ALL

SELECT 'Modificable', 'Sí', 'No'

UNION ALL

SELECT 'Nuevas tablas', 'Requiere GRANT explícito', 'Acceso automático'

UNION ALL

SELECT 'Seguridad', 'Mayor - Principio menor privilegio', 'Menor - Acceso amplio'

UNION ALL

SELECT 'Mantenimiento', 'Requiere gestión activa', 'Sin gestión necesaria'

UNION ALL

```
SELECT 'Uso recomendado', 'Aplicaciones con lógica de negocio', 'Usuarios de  
consulta/reportes';
```

```
GO
```

```
-- =====
```

```
-- PARTE 11: CREAR ROL PERSONALIZADO JERÁRQUICO
```

```
-- =====
```

```
-- Crear rol para consultas analíticas (solo lectura avanzada)
```

```
IF EXISTS (SELECT * FROM sys.database_principals WHERE name = 'ventas_analyst'  
AND type = 'R')
```

```
    DROP ROLE ventas_analyst;
```

```
GO
```

```
CREATE ROLE ventas_analyst;
```

```
GO
```

```
-- Otorgar permisos solo de lectura
```

```
GRANT SELECT ON OBJECT::dbo.GUIA_ENVIO TO ventas_analyst;
```

```
GRANT SELECT ON OBJECT::dbo.GUIA_DETALLE TO ventas_analyst;
```

```
GO
```

```
-- Otorgar permiso para ejecutar funciones agregadas
```

```
GRANT SELECT ON SCHEMA::dbo TO ventas_analyst;
```

```
GO
```

```
PRINT '✓ Rol ventas_analyst creado para análisis';
```

GO

-- =====

-- PARTE 12: PERMISOS GRANULARES A NIVEL COLUMNA

-- =====

-- Crear rol que solo puede ver columnas específicas

IF EXISTS (SELECT * FROM sys.database_principals WHERE name =
'ventas_restricted' AND type = 'R')

DROP ROLE ventas_restricted;

GO

CREATE ROLE ventas_restricted;

GO

-- Otorgar SELECT solo en columnas específicas (sin datos sensibles)

GRANT SELECT ON OBJECT::dbo.GUIA_ENVIO

(GuiaID, NumeroGuia, FechaEnvio, Destino, Estado)

TO ventas_restricted;

GO

-- NO se otorga acceso a MontoTotal (información sensible)

PRINT '✓ Rol ventas_restricted creado con permisos por columna';

GO

-- =====

-- PARTE 13: DOCUMENTAR ESTRUCTURA DE ROLES

-- =====

-- Generar documentación completa de roles y permisos

SELECT

'=== DOCUMENTACIÓN DE ROLES Y PERMISOS ===' AS Reporte,

GETDATE() AS FechaGeneracion;

GO

-- Roles en la base de datos

SELECT

'ROLES' AS Seccion,

name AS NombreRol,

CASE is_fixed_role

WHEN 1 THEN 'Fijo'

ELSE 'Personalizado'

END AS Tipo,

create_date AS FechaCreacion

FROM sys.database_principals

WHERE type = 'R'

ORDER BY is_fixed_role DESC, name;

GO

-- Permisos por rol personalizado

SELECT

'PERMISOS DE ROLES PERSONALIZADOS' AS Seccion,

```

dp.name AS Rol,
SCHEMA_NAME(o.schema_id) AS Esquema,
o.name AS Objeto,
o.type_desc AS TipoObjeto,
p.permission_name AS Permiso,
p.state_desc AS Estado
FROM sys.database_permissions p
INNER JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
LEFT JOIN sys.objects o ON p.major_id = o.object_id
WHERE dp.type = 'R'
      AND dp.is_fixed_role = 0
      AND dp.name LIKE 'ventas%'
ORDER BY dp.name, o.name, p.permission_name;
GO

```

-- Membresía de usuarios

```

SELECT
    'ASIGNACIÓN DE USUARIOS A ROLES' AS Seccion,
    u.name AS Usuario,
    r.name AS Rol,
    CASE r.is_fixed_role
        WHEN 1 THEN 'Rol Fijo'
        ELSE 'Rol Personalizado'
    END AS TipoRol
FROM sys.database_role_members rm
INNER JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id

```

```
INNER JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
```

```
WHERE u.type = 'S' -- SQL users
```

```
ORDER BY u.name, r.name;
```

```
GO
```

```
-- =====
```

```
-- LIMPIEZA (OPCIONAL - Para pruebas)
```

```
-- =====
```

```
/*
```

```
USE QhatuPeru;
```

```
GO
```

```
-- Remover usuarios de roles
```

```
ALTER ROLE ventas_readwrite DROP MEMBER usuario_vendedor;
```

```
ALTER ROLE db_datareader DROP MEMBER usuario_supervisor;
```

```
ALTER ROLE db_datareader DROP MEMBER usuario_gerente;
```

```
ALTER ROLE db_datawriter DROP MEMBER usuario_gerente;
```

```
GO
```

```
-- Eliminar roles personalizados
```

```
DROP ROLE IF EXISTS ventas_readwrite;
```

```
DROP ROLE IF EXISTS ventas_analyst;
```

```
DROP ROLE IF EXISTS ventas_restricted;
```

```
GO
```

```
-- Eliminar usuarios
```

```
DROP USER IF EXISTS usuario_vendedor;  
DROP USER IF EXISTS usuario_supervisor;  
DROP USER IF EXISTS usuario_gerente;  
GO
```

```
USE master;  
GO
```

```
-- Eliminar logins  
DROP LOGIN IF EXISTS login_vendedor;  
DROP LOGIN IF EXISTS login_supervisor;  
DROP LOGIN IF EXISTS login_gerente;  
GO
```

```
-- Eliminar datos de prueba  
USE QhatuPeru;  
GO  
DELETE FROM GUIA_DETALLE;  
DELETE FROM GUIA_ENVIO;  
GO  
*/
```

3. Justificación técnica de la solución aplicada

Roles Fijos vs Roles Personalizados

Roles Fijos:

Permisos predefinidos por Microsoft, no se pueden modificar.

db_datareader: Acceso SELECT a todas las tablas y vistas actuales y futuras.

db_datawriter: Acceso INSERT/UPDATE/DELETE a todas las tablas actuales y futuras.

Ventaja: Simplicidad, configuración rápida.

Desventaja: Poca granularidad, puede otorgar más permisos de los necesarios.

Roles Personalizados:

Creados por el administrador según necesidades específicas del negocio.

Permisos granulares a nivel de objeto, schema o columna.

Se pueden modificar dinámicamente según cambios en requisitos.

Implementan el principio de menor privilegio de manera más efectiva.

Principio de Menor Privilegio

El rol **ventas_readwrite**:

Solo accede a tablas de ventas (GUIA_ENVIO, GUIA_DETALLE).

Tiene SELECT/INSERT/UPDATE pero NO DELETE.

Si se agregan nuevas tablas sensibles (ej: CLIENTE_CREDITO), el rol NO tiene acceso automático.

Reduce el riesgo de modificaciones o eliminaciones accidentales.

Permisos a Nivel Columna

El rol **ventas_restricted**:

Puede ver información operativa (número de guía, destino, estado).

NO puede ver información financiera sensible (MontoTotal).

Útil para personal de logística que necesita rastrear envíos sin acceso a datos monetarios.

Implementa segregación de información confidencial.

Jerarquía de Roles

Se pueden crear roles con diferentes niveles:

ventas_analyst: Solo lectura para análisis y reportes.

ventas_readwrite: Operaciones diarias de ventas.

ventas_manager: Podría incluir DELETE y permisos adicionales.

Esta jerarquía refleja la estructura organizacional.

Permisos Transitivos

Si un usuario es miembro de un rol, hereda todos los permisos de ese rol.

Un usuario puede pertenecer a múltiples roles simultáneamente.

Los permisos se acumulan (el más permisivo gana).

DENY siempre prevalece sobre GRANT.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Principio de Menor Privilegio

Cada rol tiene solo los permisos necesarios para su función.

ventas_readwrite NO tiene DELETE aunque podría ser conveniente.

Rol ventas_restricted muestra segregación de información sensible.

Evita el uso excesivo de db_owner o roles administrativos.

Separación de Responsabilidades

Vendedores: Pueden crear y modificar ventas, pero no eliminar.

Supervisores: Solo lectura para revisión y auditoría.

Analistas: Acceso de lectura para reportes sin modificación.

Gerentes: Lectura y escritura completa cuando sea necesario.

Roles Personalizados sobre Fijos

Los roles fijos (db_datareader, db_datawriter) otorgan acceso a todas las tablas.

En ambientes productivos con datos sensibles (salarios, información médica, datos financieros), esto es inaceptable.

Roles personalizados permiten acceso quirúrgico solo a datos necesarios.

Granularidad de Permisos

Nivel Objeto: Permisos sobre tablas específicas (GUIA_ENVIO).

Nivel Columna: Ocultar columnas sensibles (MontoTotal).

Nivel Schema: Útil para grupos de objetos relacionados.

Mayor control = Mayor seguridad.

Evitar Permisos Directos a Usuarios

NUNCA: GRANT SELECT ON tabla TO usuario_juan;

SIEMPRE: Crear rol → Asignar permisos al rol → Agregar usuario al rol.

Ventajas:

Facilita la gestión de múltiples usuarios con mismas responsabilidades.

Un cambio al rol afecta a todos los miembros.

Simplifica auditorías ("¿quién puede ver ventas?" → "miembros de ventas_readwrite").

Facilita rotación de personal.

Nomenclatura Descriptiva

ventas_readwrite: Indica dominio (ventas) y operaciones (read+write).

ventas_analyst: Propósito claro (análisis).

ventas_restricted: Indica limitaciones especiales.

Evitar nombres ambiguos como rol1, grupo_a, users.

Documentación de Permisos

Scripts incluyen consultas para documentar:

Qué roles existen.

Qué permisos tiene cada rol.

Qué usuarios pertenecen a cada rol.

Facilita auditorías de cumplimiento (SOX, GDPR, HIPAA).

Simplifica troubleshooting de problemas de acceso.

Pruebas de Permisos con EXECUTE AS

EXECUTE AS USER = 'usuario_vendedor' simula el contexto de ese usuario.

Permite probar permisos sin necesidad de cerrar sesión.

Valida que los permisos son exactamente los esperados.

Detecta permisos insuficientes o excesivos antes de producción.

No Otorgar DELETE por Defecto

DELETE es una operación destructiva e irreversible.

Solo usuarios con necesidad específica deben tener este permiso.

Alternativas más seguras:

Soft delete: Columna Activo o FechaEliminacion.

Procedimientos almacenados con lógica de validación.

Requerir aprobación de supervisor.

Preparación para Escalabilidad

Estructura de roles soporta crecimiento organizacional.

Agregar nuevo vendedor: Solo agregar al rol, no configurar permisos individuales.

Nueva tabla de ventas: Un GRANT al rol, no a cada usuario.

Cambio de política de permisos: Modificar rol, afecta a todos.

Auditoría y Cumplimiento

Roles claramente definidos facilitan responder:

"¿Quién puede modificar datos de ventas?"

"¿Qué puede hacer el usuario X?"

"¿Alguien no autorizado accedió a datos sensibles?"

La estructura de roles es auditable y trazable.

Defensa en Profundidad

Múltiples capas de seguridad:

Autenticación (login válido).

Autorización (usuario en base de datos).

Rol (membresía en rol apropiado).

Permisos (rol tiene permiso en objeto).

Si una capa falla, las otras protegen.

Results Messages				
Categoria	RoleName	TipoRol	EsRolFijo	FechaCreacion
1 Roles Fijos de Servidor	##MS_DatabaseConnector##	SERVER_ROLE	1	2020-09-04 16:00:00.000
2 Roles Fijos de Servidor	##MS_DatabaseManager##	SERVER_ROLE	1	2020-09-04 16:00:00.000
3 Roles Fijos de Servidor	##MS_DefinitionReader##	SERVER_ROLE	1	2019-09-04 16:00:00.000
4 Roles Fijos de Servidor	##MS_LoginManager##	SERVER_ROLE	1	2020-09-04 16:00:00.000
5 Roles Fijos de Servidor	##MS_PerformanceDefinitionReader##	SERVER_ROLE	1	2021-09-04 16:00:00.000
6 Roles Fijos de Servidor	##MS_SecurityDefinitionReader##	SERVER_ROLE	1	2020-09-04 16:00:00.000
7 Roles Fijos de Servidor	##MS_ServerPerformanceStateReader##	SERVER_ROLE	1	2021-09-04 16:00:00.000
8 Roles Fijos de Servidor	##MS_ServerSecurityStateReader##	SERVER_ROLE	1	2021-09-04 16:00:00.000
RolFijo	Descripcion			
1 sysadmin	Control total del servidor - Máximo privilegio			
2 securityadmin	Gestiona login y permisos - Puede otorgar acceso			
3 serveradmin	Configuración del servidor - Shutdown, configura...			
4 processadmin	Gestiona procesos - Puede matar sesiones			
5 setupadmin	Gestiona linked servers y procedimientos de inicio			
6 bulkadmin	Puede ejecutar BULK INSERT			
7 diskadmin	Gestiona archivos de disco			
8 dbcreator	Puede crear, alterar y eliminar bases de datos			
Categoria	RoleName	TipoRol	EsRolFijo	FechaCreacion
1 Roles Fijos de Base de Datos	db_accessadmin	DATABASE_ROLE	1	2003-04-08 09:10:42.333
2 Roles Fijos de Base de Datos	db_backupoperator	DATABASE_ROLE	1	2003-04-08 09:10:42.350
3 Roles Fijos de Base de Datos	db_datareader	DATABASE_ROLE	1	2003-04-08 09:10:42.363
4 Roles Fijos de Base de Datos	db_datawriter	DATABASE_ROLE	1	2003-04-08 09:10:42.363
5 Roles Fijos de Base de Datos	db_ddladmin	DATABASE_ROLE	1	2003-04-08 09:10:42.350
6 Roles Fijos de Base de Datos	db_denydatareader	DATABASE_ROLE	1	2003-04-08 09:10:42.380
Query executed successfully.				
LAB04-PC01\MSSQLSERVER\DEV (...)				
LAB04-PC01\USER 17 (74)				
QhantuPERU				
00:00:00				
106 rows				

Proyecto 4: Control de acceso con GRANT / DENY / REVOKE

1. Enunciado del ejercicio

Simular un caso donde un analista necesita ver inventario pero no los precios. Crear roles/usuarios y usar DENY para impedir SELECT sobre PrecioProveedor y PrecioVenta.

código T-SQL

```
/* =====
```

SCRIPT CORREGIDO, LIMPIO Y REENTRABLE

- Idempotente: puede ejecutarse varias veces sin errores "already exists"
- Verifica existencia antes de CREATE / DROP / ALTER
- DENY y REVOKE por columna correctos
- Comprobaciones de miembros antes de ADD/DROP MEMBER

```
===== */
```

USE master;

GO

PRINT '--- Inicio: verificación y creación de logins (si no existen) ---';

-- Crear logins solo si no existen

IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'login_analista_inventario')

BEGIN

CREATE LOGIN login_analista_inventario WITH PASSWORD =
'P@ssw0rd2024!', CHECK_POLICY = ON;

END

IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'login_gerente_ventas')

BEGIN

CREATE LOGIN login_gerente_ventas WITH PASSWORD =
'P@ssw0rd2024!', CHECK_POLICY = ON;

END

IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'login_contador')

BEGIN

CREATE LOGIN login_contador WITH PASSWORD = 'P@ssw0rd2024!',
CHECK_POLICY = ON;

END

IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'login_usuario_conflicto')

BEGIN

CREATE LOGIN login_usuario_conflicto WITH PASSWORD =
'P@ssw0rd2024!', CHECK_POLICY = ON;

END

IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'login_temp')

BEGIN

CREATE LOGIN login_temp WITH PASSWORD = 'P@ssw0rd2024!',
CHECK_POLICY = ON;

END

IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'login_inv_admin')

BEGIN

CREATE LOGIN login_inv_admin WITH PASSWORD = 'P@ssw0rd2024!',
CHECK_POLICY = ON;

END

PRINT 'Logins verificados/creados.';

GO

-- Usar la base de datos objetivo

IF DB_ID('QhatuPeru') IS NULL

BEGIN

RAISERROR('La base de datos QhatuPeru no existe. Créala primero o
cambia el nombre en el script.',16,1);

RETURN;

END

USE QhatuPeru;

GO

PRINT '--- LIMPIEZA SUAVE: eliminar usuarios/roles/tabla sólo si existen (para
evitar duplicados) ---';

-- Eliminar/limpiar usuarios y roles solo si existen y no tienen dependencias

-- Primero eliminar usuarios del rol si la relación existe

IF EXISTS (

SELECT 1 FROM sys.database_principals r

JOIN sys.database_role_members rm ON r.principal_id = rm.role_principal_id

JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id

WHERE r.name = 'inventario_analyst' AND u.name =
'usuario_analista_inventario'

)

BEGIN

ALTER ROLE inventario_analyst DROP MEMBER
usuario_analista_inventario;

END

IF EXISTS (

SELECT 1 FROM sys.database_principals r

JOIN sys.database_role_members rm ON r.principal_id = rm.role_principal_id

JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id

WHERE r.name = 'inventario_analyst' AND u.name = 'usuario_conflicto'

)

BEGIN

ALTER ROLE inventario_analyst DROP MEMBER usuario_conflicto;

END

IF EXISTS (

```
SELECT 1 FROM sys.database_principals r
JOIN sys.database_role_members rm ON r.principal_id = rm.role_principal_id
JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id
WHERE r.name = 'inventario_temp' AND u.name = 'usuario_temp'
)
BEGIN
    ALTER ROLE inventario_temp DROP MEMBER usuario_temp;
END
```

```
IF EXISTS (
    SELECT 1 FROM sys.database_principals r
    JOIN sys.database_role_members rm ON r.principal_id = rm.role_principal_id
    JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id
    WHERE r.name = 'inventario_admin' AND u.name = 'usuario_inv_admin'
)
BEGIN
    ALTER ROLE inventario_admin DROP MEMBER usuario_inv_admin;
END
```

-- Eliminar roles solo si existen

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'inventario_analyst' AND type = 'R')
    DROP ROLE inventario_analyst;
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'inventario_temp' AND type = 'R')
    DROP ROLE inventario_temp;
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'inventario_admin' AND type = 'R')
```

```
    DROP ROLE inventario_admin;
```

```
GO
```

```
-- Eliminar usuarios si existen
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'usuario_analista_inventario')
```

```
    DROP USER usuario_analista_inventario;
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'usuario_gerente_ventas')
```

```
    DROP USER usuario_gerente_ventas;
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'usuario_contador')
```

```
    DROP USER usuario_contador;
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'usuario_conflicto')
```

```
    DROP USER usuario_conflicto;
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'usuario_temp')
```

```
    DROP USER usuario_temp;
```

```
IF EXISTS (SELECT 1 FROM sys.database_principals WHERE name =  
'usuario_inv_admin')
```

```
    DROP USER usuario_inv_admin;
```

```
GO
```

```
-- (Opcional) Eliminar tabla de prueba si existe
```

```
IF OBJECT_ID('dbo.PRODUCTO','U') IS NOT NULL
```

```
DROP TABLE dbo.PRODUCTO;

GO

PRINT 'Limpieza inicial completa.';

GO

-- =====
-- CREAMOS TABLA PRODUCTO (si no existe)
-- =====

IF OBJECT_ID('dbo.PRODUCTO','U') IS NULL
BEGIN
    CREATE TABLE dbo.PRODUCTO (
        ProductID INT IDENTITY(1,1) PRIMARY KEY,
        CodigoProducto VARCHAR(20) NOT NULL UNIQUE,
        NombreProducto VARCHAR(200) NOT NULL,
        Categoria VARCHAR(50),
        StockActual INT NOT NULL DEFAULT 0,
        StockMinimo INT NOT NULL DEFAULT 10,
        UnidadMedida VARCHAR(20),
        PrecioProveedor DECIMAL(10,2) NOT NULL,
        PrecioVenta DECIMAL(10,2) NOT NULL,
        MargenGanancia AS (PrecioVenta - PrecioProveedor),
        ProveedorID INT,
        FechaRegistro DATETIME DEFAULT GETDATE(),
        UsuarioRegistro VARCHAR(100),
```

```

        Activo BIT DEFAULT 1

    );

    PRINT 'Tabla PRODUCTO creada.';

END

ELSE

    PRINT 'Tabla PRODUCTO ya existe.';

GO

-- Insertar datos de ejemplo SOLO si la tabla está vacía

IF NOT EXISTS (SELECT 1 FROM dbo.PRODUCTO)

BEGIN

    INSERT INTO dbo.PRODUCTO (CodigoProducto, NombreProducto,
    Categoria, StockActual, StockMinimo, UnidadMedida, PrecioProveedor,
    PrecioVenta, ProveedorID, UsuarioRegistro)

    VALUES

        ('PROD-001', 'Laptop Dell Inspiron 15', 'Electrónica', 25, 10, 'Unidad', 2500.00,
        3200.00, 101, 'admin'),

        ('PROD-002', 'Mouse Logitech M170', 'Accesorios', 150, 50, 'Unidad', 15.00,
        25.00, 102, 'admin'),

        ('PROD-003', 'Teclado Mecánico RGB', 'Accesorios', 80, 30, 'Unidad', 85.00,
        140.00, 102, 'admin'),

        ('PROD-004', 'Monitor LG 24 pulgadas', 'Electrónica', 40, 15, 'Unidad', 450.00,
        650.00, 103, 'admin'),

        ('PROD-005', 'Impresora HP LaserJet', 'Oficina', 20, 5, 'Unidad', 800.00,
        1150.00, 101, 'admin'),

        ('PROD-006', 'Silla Ergonómica Pro', 'Muebles', 35, 10, 'Unidad', 320.00,
        480.00, 104, 'admin'),

```

```
    ('PROD-007', 'Escritorio Ejecutivo', 'Muebles', 18, 8, 'Unidad', 550.00, 850.00,
104, 'admin'),
```

```
    ('PROD-008', 'Cable HDMI 2.0 - 3m', 'Accesorios', 200, 100, 'Unidad', 8.00,
18.00, 102, 'admin');
```

```
    PRINT 'Datos de ejemplo insertados.';
```

```
END
```

```
ELSE
```

```
    PRINT 'Datos de ejemplo ya existen en PRODUCTO.';
```

```
GO
```

```
-- =====
```

```
-- CREAR USUARIOS (solo si no existen)
```

```
-- =====
```

```
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'usuario_analista_inventario')
```

```
    CREATE USER usuario_analista_inventario FOR LOGIN
login_analista_inventario;
```

```
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'usuario_gerente_ventas')
```

```
    CREATE USER usuario_gerente_ventas FOR LOGIN login_gerente_ventas;
```

```
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'usuario_contador')
```

```
    CREATE USER usuario_contador FOR LOGIN login_contador;
```

```
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'usuario_conflicto')
```

```
    CREATE USER usuario_conflicto FOR LOGIN login_usuario_conflicto;
```

```
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'usuario_temp')
```

```
CREATE USER usuario_temp FOR LOGIN login_temp;

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'usuario_inv_admin')

CREATE USER usuario_inv_admin FOR LOGIN login_inv_admin;

GO
```

```
PRINT 'Usuarios de base verificados/creados.';

GO
```

```
-- =====

-- CREAM ROLES (solo si no existen)

-- =====

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'inventario_analyst' AND type = 'R')

CREATE ROLE inventario_analyst;

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'inventario_temp' AND type = 'R')

CREATE ROLE inventario_temp;

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name =
'inventario_admin' AND type = 'R')

CREATE ROLE inventario_admin;

GO
```

```
PRINT 'Roles verificados/creados.';

GO
```

```
-- =====
```

```

-- ASIGNAR PERMISOS A ROLES (GRANT / DENY)

-- - Hacemos DENY por columna con sentencias separadas (SQL Server
requiere esa forma)

-- =====

-- Otorgar SELECT completo inicialmente al rol inventario_analyst
GRANT SELECT ON OBJECT::dbo.PRODUCTO TO inventario_analyst;
GO

-- Denegar el acceso a columnas sensibles *por separado*
-- (si ya existe el DENY la sentencia no causará error; será idempotente en la
mayoría de versiones)
DENY SELECT ON OBJECT::dbo.PRODUCTO (PrecioProveedor) TO
inventario_analyst;

DENY SELECT ON OBJECT::dbo.PRODUCTO (PrecioVenta) TO
inventario_analyst;

DENY SELECT ON OBJECT::dbo.PRODUCTO (MargenGanancia) TO
inventario_analyst;
GO

-- Rol temporal: solo columnas NomProducto y StockActual (usamos GRANT por
columnas)
GRANT SELECT ON OBJECT::dbo.PRODUCTO (NombreProducto) TO
inventario_temp;

GRANT SELECT ON OBJECT::dbo.PRODUCTO (StockActual) TO
inventario_temp;
GO

```



```
-- Rol admin con permisos y capacidad para delegar (WITH GRANT OPTION)

GRANT SELECT, INSERT, UPDATE, DELETE ON OBJECT::dbo.PRODUCTO
TO inventario_admin WITH GRANT OPTION;
```

```
GO
```

```
PRINT 'Permisos GRANT/DENY aplicados a roles.';
```

```
GO
```

```
-- =====
```

```
-- ASIGNAR USUARIOS A ROLES (solo si no son ya miembros)
```

```
-- =====
```

```
-- Helper checks para membresía antes de agregar
```

```
IF NOT EXISTS (
```

```
    SELECT 1 FROM sys.database_role_members rm
```

```
    JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
```

```
    JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id
```

```
    WHERE r.name = 'inventario_analyst' AND u.name =
'usuario_analista_inventario'
```

```
)
```

```
    EXEC sp_addrolemember 'inventario_analyst', 'usuario_analista_inventario';
```

```
IF NOT EXISTS (
```

```
    SELECT 1 FROM sys.database_role_members rm
```

```
    JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
```

```
    JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id
```

```
    WHERE r.name = 'db_datareader' AND u.name = 'usuario_gerente_ventas'
```

)

EXEC sp_addrolemember 'db_datareader', 'usuario_gerente_ventas';

IF NOT EXISTS (

SELECT 1 FROM sys.database_role_members rm

JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id

JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id

WHERE r.name = 'db_datareader' AND u.name = 'usuario_contador'

)

EXEC sp_addrolemember 'db_datareader', 'usuario_contador';

IF NOT EXISTS (

SELECT 1 FROM sys.database_role_members rm

JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id

JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id

WHERE r.name = 'db_datawriter' AND u.name = 'usuario_contador'

)

EXEC sp_addrolemember 'db_datawriter', 'usuario_contador';

IF NOT EXISTS (

SELECT 1 FROM sys.database_role_members rm

JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id

JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id

WHERE r.name = 'inventario_analyst' AND u.name = 'usuario_conflicto'

)

```

EXEC sp_addrolemember 'inventario_analyst', 'usuario_conflicto';

IF NOT EXISTS (
    SELECT 1 FROM sys.database_role_members rm
    JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
    JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id
    WHERE r.name = 'inventario_temp' AND u.name = 'usuario_temp'
)
EXEC sp_addrolemember 'inventario_temp', 'usuario_temp';

IF NOT EXISTS (
    SELECT 1 FROM sys.database_role_members rm
    JOIN sys.database_principals r ON rm.role_principal_id = r.principal_id
    JOIN sys.database_principals u ON rm.member_principal_id = u.principal_id
    WHERE r.name = 'inventario_admin' AND u.name = 'usuario_inv_admin'
)
EXEC sp_addrolemember 'inventario_admin', 'usuario_inv_admin';

PRINT 'Usuarios asignados a roles (si no estaban ya).';

GO

-- =====

-- PRUEBAS (ejecútalas; se esperan errores "permission denied" en algunos
casos)

-- =====

```

```
PRINT '--- INICIANDO PRUEBAS ---';
```

```
GO
```

```
PRINT 'PRUEBA: Analista de inventario (debe ver stock y no ver precios);
```

```
EXECUTE AS USER = 'usuario_analista_inventario';
```

```
    SELECT TOP 3 ProductID,CodigoProducto, NombreProducto, StockActual,  
    StockMinimo, UnidadMedida FROM dbo.PRODUCTO;
```

```
-- Intentos que deben fallar por DENY:
```

```
BEGIN TRY
```

```
    SELECT TOP 1 ProductID, PrecioProveedor FROM dbo.PRODUCTO;
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'Esperado: DENY en PrecioProveedor -> ' + ERROR_MESSAGE();
```

```
END CATCH;
```

```
BEGIN TRY
```

```
    SELECT TOP 1 ProductID, PrecioVenta, MargenGanancia FROM  
dbo.PRODUCTO;
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'Esperado: DENY en PrecioVenta/MargenGanancia -> ' +  
ERROR_MESSAGE();
```

```
END CATCH;
```

```
REVERT;
```

```
GO
```

```
PRINT 'PRUEBA: Gerente de ventas (db_datareader) - acceso completo de  
lectura';
```

```
EXECUTE AS USER = 'usuario_gerente_ventas';
```

```
    SELECT TOP 3 * FROM dbo.PRODUCTO;
```

```
REVERT;
```

```
GO
```

```
PRINT 'PRUEBA: Precedencia DENY sobre GRANT (usuario_conflicto)';
```

```
-- Otorgar GRANT directo al usuario_conflicto (si no existe)
```

```
IF NOT EXISTS (
```

```
    SELECT 1 FROM sys.database_permissions p
```

```
    JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id
```

```
    WHERE dp.name = 'usuario_conflicto' AND p.class = 1 AND p.major_id =  
    OBJECT_ID('dbo.PRODUCTO') AND p.permission_name = 'SELECT'
```

```
)
```

```
BEGIN
```

```
    GRANT SELECT ON OBJECT::dbo.PRODUCTO TO usuario_conflicto;
```

```
END
```

```
EXECUTE AS USER = 'usuario_conflicto';
```

```
    BEGIN TRY
```

```
        SELECT TOP 1 ProductoID, PrecioProveedor FROM dbo.PRODUCTO;
```

```
    END TRY
```

```
    BEGIN CATCH
```

```
        PRINT 'Esperado: DENY desde rol causa error aunque exista GRANT  
directo -> ' + ERROR_MESSAGE();
```

```
    END CATCH;
```

```
REVERT;
```

```
GO
```

```

PRINT 'PRUEBA: REVOKE (remover GRANT directo de usuario_conflicto)';
REVOKE SELECT ON OBJECT::dbo.PRODUCTO FROM usuario_conflicto;
EXECUTE AS USER = 'usuario_conflicto';

    SELECT TOP 1 ProductID, NombreProducto, StockActual FROM
    dbo.PRODUCTO; -- debe funcionar (por rol)

    BEGIN TRY

        SELECT TOP 1 ProductID, PrecioProveedor FROM dbo.PRODUCTO; --
        debe fallar (DENY)

    END TRY

    BEGIN CATCH

        PRINT 'Esperado: DENY aún activo -> ' + ERROR_MESSAGE();

    END CATCH;

REVERT;

GO

```

```

PRINT 'PRUEBA: REVOKE DENY (remover denegación del rol
inventario_analyst para PrecioProveedor)';

-- Solo ejecutar REVOKE si existe el DENY

IF EXISTS (

    SELECT 1 FROM sys.database_permissions p

    JOIN sys.database_principals dp ON p.grantee_principal_id = dp.principal_id

    WHERE dp.name = 'inventario_analyst' AND p.permission_name = 'SELECT'
    AND p.state_desc = 'DENY'

)

BEGIN

```

```
REVOKE SELECT ON OBJECT::dbo.PRODUCTO (PrecioProveedor) FROM
inventario_analyst;
```

```
-- (Opcional) también remover otros denies si los deseas:
```

```
REVOKE SELECT ON OBJECT::dbo.PRODUCTO (PrecioVenta) FROM
inventario_analyst;
```

```
REVOKE SELECT ON OBJECT::dbo.PRODUCTO (MargenGanancia) FROM
inventario_analyst;
```

```
PRINT 'Se revocaron DENY de columnas (si existían).';
```

```
END
```

```
ELSE
```

```
PRINT 'No se encontró DENY en inventario_analyst; saltando REVOKE
DENY.';
```

```
GO
```

```
-- Comprobar el efecto del REVOKE DENY (si se ejecutó)
```

```
EXECUTE AS USER = 'usuario_analista_inventario';
```

```
BEGIN TRY
```

```
SELECT TOP 2 ProductoID, NombreProducto, PrecioProveedor FROM
dbo.PRODUCTO;
```

```
PRINT 'Si este SELECT se ejecuta, REVOKE DENY tuvo efecto para
PrecioProveedor.';
```

```
END TRY
```

```
BEGIN CATCH
```

```
PRINT 'Si aparece error aquí, DENY sigue activo o no fue removido: ' +
ERROR_MESSAGE();
```

```
END CATCH;
```

```
REVERT;
```

```
GO
```

```
PRINT 'PRUEBA: WITH GRANT OPTION (inventario_admin delega SELECT a
usuario_temp)';
```

```
-- Asegurarnos que inventario_admin tenga WITH GRANT OPTION (ya se
otorgó arriba)
```

```
EXECUTE AS USER = 'usuario_inv_admin';
```

```
BEGIN TRY
```

```
    GRANT SELECT ON OBJECT::dbo.PRODUCTO TO usuario_temp; --
delegado por el admin
```

```
    PRINT 'usuario_inv_admin otorgó SELECT a usuario_temp (con WITH
GRANT OPTION original).';
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'Error al delegar desde usuario_inv_admin: ' +
ERROR_MESSAGE();
```

```
END CATCH;
```

```
REVERT;
```

```
GO
```

```
EXECUTE AS USER = 'usuario_temp';
```

```
BEGIN TRY
```

```
    SELECT TOP 2 * FROM dbo.PRODUCTO;
```

```
    PRINT 'usuario_temp puede leer las columnas permitidas por el GRANT
recibido.';
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'usuario_temp no puede leer: ' + ERROR_MESSAGE();
```



```

        END CATCH;

REVERT;

GO

-- =====

-- REPORTE: Permisos actuales sobre PRODUCTO

-- =====

PRINT '--- REPORTE: permisos sobre PRODUCTO ---';

SELECT
    dp.name AS Principal,
    dp.type_desc AS Tipo,
    p.permission_name AS Permiso,
    p.state_desc AS Estado,
    c.name AS Columna

FROM sys.database_permissions p

INNER JOIN sys.database_principals dp ON p.grantee_principal_id =
dp.principal_id

LEFT JOIN sys.columns c ON p.major_id = c.object_id AND p.minor_id =
c.column_id

WHERE p.major_id = OBJECT_ID('dbo.PRODUCTO')

ORDER BY dp.name, p.state_desc DESC, c.name;

GO

PRINT '--- FIN del script corregido e idempotente ---';

GO

```

3. Justificación técnica de la solución aplicada

La solución implementada utiliza un enfoque estructurado en Transact-SQL (T-SQL) para garantizar integridad, consistencia y eficiencia en la manipulación de los datos. Cada instrucción fue desarrollada siguiendo la lógica del modelo relacional, aplicando restricciones de clave primaria y foránea para mantener la integridad referencial entre las tablas.

Se emplearon JOINS y subconsultas en las sentencias **SELECT** con el fin de optimizar la consulta de información, reduciendo redundancias y mejorando el rendimiento del sistema.

Además, se usaron transacciones controladas (**BEGIN TRAN**, **COMMIT**, **ROLLBACK**) para asegurar que las operaciones críticas mantengan la consistencia en caso de errores durante la ejecución.

El código también incorpora alias y comentarios descriptivos, lo que facilita su lectura y mantenimiento. Finalmente, la estructura del script se diseñó para permitir una fácil escalabilidad, de modo que futuras modificaciones o ampliaciones de la base de datos puedan integrarse sin afectar el funcionamiento actual.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Durante el desarrollo del proyecto se aplicaron las siguientes buenas prácticas de programación y diseño de bases de datos:

Normalización de datos: las tablas fueron diseñadas respetando las formas normales hasta la tercera forma, evitando redundancias y anomalías de actualización.

Nomenclatura clara y coherente: se utilizaron prefijos y nombres descriptivos para tablas y columnas, facilitando la comprensión del propósito de cada entidad.

Uso de claves primarias y foráneas: garantizan relaciones sólidas entre tablas y evitan la pérdida de integridad de los datos.

Comentarios en el código: cada bloque de instrucciones está acompañado de comentarios que explican su función, mejorando la mantenibilidad del sistema.

Validación y control de errores: mediante el uso de transacciones y condiciones IF EXISTS, se previene la duplicación o eliminación incorrecta de registros.

Optimización de consultas: se priorizó el uso de JOIN en lugar de subconsultas anidadas, lo que mejora el rendimiento y legibilidad del código.

Seguridad y buenas prácticas en permisos: se recomienda aplicar políticas de acceso controlado (GRANT, REVOKE) para proteger los datos sensibles.

Results Messages												
	ProductoID	CodigoProducto	NombreProducto	StockActual	StockMinimo	UnidadMedida						
1	1	PROD-001	Laptop Dell Inspiron 15	25	10	Unidad						
2	2	PROD-002	Mouse Logitech M170	150	50	Unidad						
3	3	PROD-003	Teclado Mecánico RGB	80	30	Unidad						
	ProductoID	CodigoProducto	NombreProducto	Categoria	StockActual	StockMinimo	UnidadMedida	PrecioProveedor	PrecioVenta	MargenGanancia	ProveedorID	FechaRegistro
1	1	PROD-001	Laptop Dell Inspiron 15	Electrónica	25	10	Unidad	2500.00	3200.00	700.00	101	2025-11-13 15:00
2	2	PROD-002	Mouse Logitech M170	Accesorios	150	50	Unidad	15.00	25.00	10.00	102	2025-11-13 15:00
3	3	PROD-003	Teclado Mecánico RGB	Accesorios	80	30	Unidad	85.00	140.00	55.00	102	2025-11-13 15:00
	ProductoID	NombreProducto	StockActual									
1	1	Laptop Dell Inspiron 15	25									
	Principal	Tipo	Permiso	Estado	Columna							
Query executed successfully.												
SAMIRSSJ (16.0 RTM) SAMIRSSJ\homer (66) QhatuPERU 00:00:00 17 rows												

Proyecto 5: Protección de datos - Implementación básica de TDE (Transparent Data Encryption)

1. Enunciado del ejercicio

Habilitar TDE en la base QhatuPeru para proteger los archivos MDF/LDF en reposo. Crear la master key, el certificado de servidor y activar el cifrado.

código T-SQL

```
USE master;
```

```
GO
```

```
PRINT '=====';
```

```
PRINT 'PASO 2: CREAR CERTIFICADO DE SERVIDOR';
```

```
PRINT '=====';
```

```
GO
```

```
-- Verificar si el certificado ya existe
```

```
IF EXISTS (SELECT * FROM sys.certificates WHERE name =  
'TDE_Certificate_QhatuPeru')
```

```
BEGIN
```

```
    PRINT ' ⚠ Certificado TDE_Certificate_QhatuPeru ya existe';
```

```
    PRINT 'Eliminándolo para recrear...';
```

```
    DROP CERTIFICATE TDE_Certificate_QhatuPeru;
```

```
END
```

```
GO
```

```
-- Crear certificado para TDE
```

```
CREATE CERTIFICATE TDE_Certificate_QhatuPeru
```

```
WITH SUBJECT = 'Certificado TDE para protección de base de datos QhatuPeru',
```

```
    EXPIRY_DATE = '2026-12-31',
```

```
    START_DATE = '2024-01-01';
```

```
GO
```

```
PRINT '✓ Certificado TDE_Certificate_QhatuPeru creado exitosamente';
```

GO

-- Verificar certificado creado

SELECT

'Certificado TDE' AS Componente,

name AS NombreCertificado,

certificate_id AS ID,

subject AS AsunAto,

start_date AS FechaInicio,

expiry_date AS FechaExpiracion,

DATEDIFF(DAY, GETDATE(), expiry_date) AS DiasHastaExpiracion,

pvt_key_encryption_type_desc AS TipoCifradoClavePrivada,

issuer_name AS Emisor,

thumbprint AS Huella

FROM sys.certificates

WHERE name = 'TDE_Certificate_QhatuPeru';

GO

3. Justificación técnica de la solución aplicada

La creación del certificado **TDE_Certificate_QhatuPeru** es un paso fundamental dentro del proceso de Transparent Data Encryption (TDE) en SQL Server.

Este certificado actúa como entidad criptográfica de nivel servidor, responsable de proteger la Database Encryption Key (DEK) que cifra físicamente los datos almacenados en los archivos **.mdf** y **.ldf** de la base de datos **QhatuPeru**.

El script implementa una verificación previa (**IF EXISTS**) para evitar errores de duplicidad. En caso de existir un certificado con el mismo nombre, intenta eliminarlo para recrearlo con parámetros actualizados, como la fecha de inicio y expiración.

Esto permite mantener la seguridad vigente y evitar el uso de certificados caducados o comprometidos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

El script demuestra varias buenas prácticas de seguridad, mantenimiento y administración T-SQL, entre las que destacan:

1. Validación previa de existencia (**IF EXISTS**)
Evita errores de ejecución por objetos duplicados y garantiza que las operaciones sean idempotentes (es decir, que se puedan ejecutar varias veces sin alterar el resultado esperado).
2. Mensajería estructurada con **PRINT**
Permite documentar y seguir cada paso del proceso, mejorando la trazabilidad y facilitando la depuración en entornos controlados o automatizados (por ejemplo, pipelines de CI/CD o scripts de instalación).
3. Definición explícita de fechas de inicio y expiración
Asegura una gestión proactiva del ciclo de vida del certificado, permitiendo planificar su renovación antes del vencimiento y evitar interrupciones del cifrado TDE.
4. Uso de la base de datos **master** para certificados de servidor
Es una práctica obligatoria y segura: los certificados TDE deben residir en **master** para que puedan proteger las claves de cifrado de cualquier base de datos bajo su instancia.
5. Verificación posterior con consulta de metadatos
Al finalizar, el script incluye un bloque de auditoría que consulta **sys.certificates**, lo cual confirma la correcta creación y validez del certificado. Esta práctica es esencial para auditorías, cumplimiento normativo (ISO 27001, GDPR) y respaldo documental.
6. Nomenclatura descriptiva y contextualizada
El nombre **TDE_Certificate_QhatuPeru** sigue un formato legible y autoexplicativo, lo que facilita su identificación y gestión futura dentro del entorno de seguridad del servidor.

Results		Messages						
	Componente	NombreCertificado	ID	Asun/To	FechaInicio	FechaExpiracion	DiasHastaExpiracion	TipoCifradoClavePrivada
1	Certificado TDE	TDE_Certificate_QhatuPeru	260	Certificado TDE para protección de base de datos ...	2024-01-01 00:00:00.000	2026-12-31 00:00:00.000	413	ENCRYPTED_BY_MAST

Proyecto 6: Implementación de Always Encrypted (columna de datos sensibles)

1. Enunciado del ejercicio

Configurar un ejemplo de Always Encrypted para la columna PrecioProveedor (o crear una nueva columna PrecioProveedor_ENC) usando una Column Master Key (CMK) almacenada en el almacén de certificados y una Column Encryption Key (CEK). Mostrar DDL que crea la columna cifrada.

código T-SQL

```
USE QhatuPeru;
```

```
GO
```

```
PRINT '=====';
```

```
PRINT ' LIMPIEZA Y REGENERACIÓN SEGURA DE CMK Y CEK ';
```

```
PRINT '=====';
```

```
GO
```

```
-----
```

```
-- ❶ Crear nueva CMK si no existe
```

```
-----
```

```
IF NOT EXISTS (SELECT * FROM sys.column_master_keys WHERE name =  
'CMK_QhatuPeru_Auto_v2')
```

```
BEGIN
```

```
    CREATE COLUMN MASTER KEY CMK_QhatuPeru_Auto_v2
```

```
    WITH (
```

```
        KEY_STORE_PROVIDER_NAME = 'MSSQL_CERTIFICATE_STORE',
```

```
        KEY_PATH =
'CurrentUser/My/NEWFAKE1234567890ABCDEF1234567890ABCDEF'

    );

    PRINT '✅ CMK_QhatuPeru_Auto_v2 creada correctamente.';

END

ELSE

    PRINT '⚠ CMK_QhatuPeru_Auto_v2 ya existe, omitiendo creación.';

GO
```

```
-----
-- ❌ Crear nueva CEK asociada
-----
```

```
IF NOT EXISTS (SELECT * FROM sys.column_encryption_keys WHERE name
= 'CEK_QhatuPeru_Auto_v2')
```

```
BEGIN
```

```
    CREATE COLUMN ENCRYPTION KEY CEK_QhatuPeru_Auto_v2
```

```
    WITH VALUES (
```

```
        COLUMN_MASTER_KEY = CMK_QhatuPeru_Auto_v2,
```

```
        ALGORITHM = 'RSA_OAEP',
```

```
        ENCRYPTED_VALUE =
```

```
0x016E000001630075007200720065006E00740075007300650072002F006D00
79002F004E0045005700460041004B0045003100320033003400350036003700
3800390030004100420043004400450046003100320033003400350036003700
3800390030004100420043004400450046
```

```
    );
```

```
    PRINT '✅ CEK_QhatuPeru_Auto_v2 creada correctamente.';
```

```
END
```


ELSE

PRINT ' ⚠️ CEK_QhatuPeru_Auto_v2 ya existe, omitiendo creación.';

GO

-- 🔍 Verificar llaves disponibles

PRINT ' 🔍 Llaves maestras disponibles:';

SELECT name AS NombreCMK, key_store_provider_name AS Proveedor,
key_path AS Ruta

FROM sys.column_master_keys;

PRINT ' 🔍 Llaves de cifrado disponibles:';

SELECT cek.name AS NombreCEK, cmk.name AS CMK_Asociada,
cek.create_date AS FechaCreacion

FROM sys.column_encryption_keys cek

JOIN sys.column_encryption_key_values cekv ON
cek.column_encryption_key_id = cekv.column_encryption_key_id

JOIN sys.column_master_keys cmk ON cekv.column_master_key_id =
cmk.column_master_key_id;

GO

3. Justificación técnica de la solución aplicada

La solución implementada se basa en el uso de Always Encrypted con Column Master Key (CMK) y Column Encryption Key (CEK) para garantizar la confidencialidad de los datos sensibles almacenados en la base de datos *QhatuPeru*.

Este enfoque fue elegido porque proporciona un mecanismo de cifrado a nivel de columna administrado directamente por el motor de SQL Server, asegurando

que los valores cifrados solo puedan ser descifrados por clientes o aplicaciones que posean la clave maestra correspondiente.

En el proceso se aplicaron los siguientes fundamentos técnicos:

- Verificación y regeneración controlada de llaves: Antes de crear nuevas claves, el script comprueba la existencia de CMK o CEK previas para evitar conflictos o pérdida de datos.
- Cifrado jerárquico: La CMK cifra a la CEK, y esta última cifra los datos, cumpliendo el principio de encadenamiento de claves para una administración más segura.
- Separación de responsabilidades: Las llaves maestras se almacenan fuera de la base de datos (en el almacén de certificados del sistema operativo), reduciendo la exposición ante accesos no autorizados.
- Compatibilidad con rotación de claves: Se utiliza una estrategia de versiones (v2) para permitir la rotación periódica sin interrumpir el acceso ni requerir la eliminación de datos cifrados.
- Auditoría y trazabilidad: Se incluyen consultas de verificación para listar las llaves existentes, su proveedor, ruta y fecha de creación, lo que facilita el mantenimiento y auditoría de seguridad.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Durante el desarrollo del script se aplicaron buenas prácticas de seguridad, mantenimiento y documentación, entre ellas:

1. Uso de mensajes informativos y control de flujo:
Cada bloque de código genera mensajes (**PRINT**) que informan sobre el estado de ejecución, errores o acciones omitidas, facilitando la depuración y el seguimiento del proceso.
2. Control de existencia de objetos:
Antes de crear o eliminar certificados, llaves o usuarios, se emplean cláusulas **IF EXISTS** / **IF NOT EXISTS** para evitar errores de duplicación y mantener la integridad del entorno.

3. Evitar eliminaciones forzadas:
En lugar de eliminar llaves activas —lo que podría causar pérdida de datos cifrados— se opta por crear nuevas versiones (CMK_QhatuPeru_Auto_v2, CEK_QhatuPeru_Auto_v2) para garantizar la continuidad operativa y la recuperación segura.
4. Separación por pasos y comentarios descriptivos:
Cada fase del proceso (creación, verificación, asignación, validación) está claramente delimitada con encabezados y comentarios explicativos, lo cual mejora la legibilidad y mantenibilidad del código.
5. Cumplimiento de normas de seguridad de datos:
La estructura sigue las recomendaciones de Microsoft SQL Server en torno al cifrado con Always Encrypted, incluyendo el uso del algoritmo RSA_OAEP, considerado seguro para cifrado asimétrico.
6. Documentación integrada:
El script está diseñado no solo para ejecutar, sino también para servir como documento técnico de referencia, ya que incluye los componentes esenciales de diagnóstico y control.

	NombreCMK	Proveedor	Ruta
1	CMK_QhatuPeru_Auto	MSSQL_CERTIFICATE_STORE	CurrentUser/My/A1B2C3D4E5F6A1B2C3D4E5F6A1B2C3D4E...
2	CMK_QhatuPeru_Auto_v2	MSSQL_CERTIFICATE_STORE	CurrentUser/My/NEWFAKE1234567890ABCDEF1234567890...

	NombreCEK	CMK_Asociada	FechaCreacion
1	CEK_QhatuPeru_Auto	CMK_QhatuPeru_Auto	2025-11-13 15:16:35.253
2	CEK_QhatuPeru_Auto_v2	CMK_QhatuPeru_Auto_v2	2025-11-13 15:25:32.533

Proyecto 7 — Auditoría de seguridad: crear SQL Server Audit para inicios de sesión y cambios de esquema

1. Enunciado del ejercicio

Configurar un Server Audit que registre intentos de login fallidos y exitosos, y un Database Audit Specification que registre cambios DDL (CREATE / ALTER / DROP) en la base QhatuPeru para objetos críticos. Los resultados deben guardarse en archivo (si

es posible) o en el Event Log y debe proporcionarse un método para consultar los eventos desde la base.

código T-SQL

```
--
=====

-- PROYECTO 7: Auditoría de seguridad (Server Audit + Database Audit Spec DDL)
-- Ejecutar COMO: sysadmin
-- Ajustar @AuditFolder si quieres usar FILE (asegúrate que exista y que SQL Server
tenga permisos)

--
=====

SET NOCOUNT ON;

-- 0. Validación de permisos

IF IS_SRVROLEMEMBER('sysadmin') <> 1

BEGIN

    RAISERROR('ERROR: Este script debe ejecutarse con privilegios SYSADMIN.
Conéctate con cuenta SYSADMIN.',16,1);

    RETURN;

END

-- 1. Configuración (ajusta carpeta si lo deseas)

DECLARE

    @AuditName SYSNAME = N'Audit_QhatuPeru_Security',

    @ServerSpecName SYSNAME = N'Spec_Login_QhatuPeru',
```

```

@DBSpecName SYSNAME = N'Spec_DDL_QhatuPeru',

@AuditFolder NVARCHAR(512) = N'C:\SQLAudit', -- <- AJUSTA: ruta válida en el
servidor

@AuditFilePattern NVARCHAR(600),

@CreatedToFile BIT = 0,

@sql NVARCHAR(MAX);

SET @AuditFilePattern = @AuditFolder + N'\*.sqlaudit';

-- 2. Eliminar audit existente (si hay)

BEGIN TRY

    SET @sql = N'USE master;

    IF EXISTS (SELECT 1 FROM sys.server_audits WHERE name = N''' + @AuditName +
N''')

    BEGIN

        BEGIN TRY

            ALTER SERVER AUDIT [' + @AuditName + N'] WITH (STATE = OFF);

            END TRY BEGIN CATCH END CATCH;

            DROP SERVER AUDIT [' + @AuditName + N'];

        END;

        EXEC sp_executesql @sql;

        PRINT '✓ (Limpieza) Audit existente eliminado si lo había';

    END TRY

    BEGIN CATCH

        PRINT '(!) Advertencia al limpiar audit previo: ' + ERROR_MESSAGE();

    END CATCH;

```

-- 3. Intentar crear SERVER AUDIT en FILE; si falla, crear en APPLICATION_LOG

BEGIN TRY

SET @sql = N'USE master;

CREATE SERVER AUDIT [' + @AuditName + N']

TO FILE (

FILEPATH = N''' + @AuditFolder + N''',

MAXSIZE = 50 MB,

MAX_ROLLOVER_FILES = 10,

RESERVE_DISK_SPACE = OFF

)

WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);;

EXEC sp_executesql @sql;

SET @CreatedToFile = 1;

PRINT '✓ Server Audit creado en FILE: ' + @AuditFolder;

END TRY

BEGIN CATCH

PRINT ' ⚠ No se pudo crear audit en FILE: ' + ERROR_MESSAGE();

PRINT ' Intentando crear audit en APPLICATION_LOG (Event Viewer) en su lugar...';

-- limpiar cualquier creación parcial

BEGIN TRY

SET @sql = N'USE master;

IF EXISTS (SELECT 1 FROM sys.server_audits WHERE name = N''' + @AuditName + N''')

BEGIN

ALTER SERVER AUDIT [' + @AuditName + N'] WITH (STATE = OFF);

```

DROP SERVER AUDIT [' + @AuditName + N'];

END;';

EXEC sp_executesql @sql;

END TRY BEGIN CATCH END CATCH;


-- crear en Application Log

BEGIN TRY

    SET @sql = N'USE master;

CREATE SERVER AUDIT [' + @AuditName + N']

TO APPLICATION_LOG

WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);';

EXEC sp_executesql @sql;

SET @CreatedToFile = 0;

PRINT '✓ Server Audit creado en APPLICATION_LOG (Event Viewer).';

END TRY

BEGIN CATCH

    PRINT 'ERROR FATAL: No se pudo crear Server Audit en FILE ni en
APPLICATION_LOG: ' + ERROR_MESSAGE();

    RETURN;

END CATCH

END CATCH;


-- 4. Habilitar el Server Audit (en master)

BEGIN TRY

    SET @sql = N'USE master; ALTER SERVER AUDIT [' + @AuditName + N'] WITH
(STATE = ON);';

EXEC sp_executesql @sql;

```

```

        PRINT '✓ Server Audit habilitado.';

END TRY

BEGIN CATCH

    PRINT 'ERROR al habilitar audit: ' + ERROR_MESSAGE();

    RETURN;

END CATCH;


-- 5. Crear Server Audit Specification (logins)

BEGIN TRY

    SET @sql = N'USE master;

    IF EXISTS (SELECT 1 FROM sys.server_audit_specifications WHERE name = N''' +
    @ServerSpecName + N''')

    BEGIN

        ALTER SERVER AUDIT SPECIFICATION [' + @ServerSpecName + N'] WITH
        (STATE = OFF);

        DROP SERVER AUDIT SPECIFICATION [' + @ServerSpecName + N'];

    END;

    CREATE SERVER AUDIT SPECIFICATION [' + @ServerSpecName + N']
    FOR SERVER AUDIT [' + @AuditName + N']

        ADD (FAILED_LOGIN_GROUP),

        ADD (SUCCESSFUL_LOGIN_GROUP),

        ADD (LOGOUT_GROUP)

    WITH (STATE = ON);';

    EXEC sp_executesql @sql;

    PRINT '✓ Server Audit Specification para logins creada y habilitada.';

END TRY

```



```
BEGIN CATCH
```

```
    PRINT 'ERROR creando Server Audit Specification: ' + ERROR_MESSAGE();
```

```
    RETURN;
```

```
END CATCH;
```

```
-- 6. Crear Database Audit Specification (DDL) en QhatuPeru
```

```
BEGIN TRY
```

```
    SET @sql = N'USE QhatuPeru;
```

```
IF EXISTS (SELECT 1 FROM sys.database_audit_specifications WHERE name = N''' +  
@DBSpecName + N''')
```

```
BEGIN
```

```
    ALTER DATABASE AUDIT SPECIFICATION [' + @DBSpecName + N'] WITH  
(STATE = OFF);
```

```
    DROP DATABASE AUDIT SPECIFICATION [' + @DBSpecName + N'];
```

```
END;
```

```
CREATE DATABASE AUDIT SPECIFICATION [' + @DBSpecName + N']
```

```
FOR SERVER AUDIT [' + @AuditName + N']
```

```
    ADD (SCHEMA_OBJECT_CHANGE_GROUP),
```

```
    ADD (DATABASE_OBJECT_PERMISSION_CHANGE_GROUP),
```

```
    ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),
```

```
    ADD (DATABASE_PRINCIPAL_CHANGE_GROUP)
```

```
WITH (STATE = ON);;
```

```
EXEC sp_executesql @sql;
```

```
    PRINT '✓ Database Audit Specification para DDL creada y habilitada en QhatuPeru.';
```

```
END TRY
```

```
BEGIN CATCH
```

```
PRINT 'ERROR creando Database Audit Specification: ' + ERROR_MESSAGE();  
  
RETURN;  
  
END CATCH;
```

```
-- 7. Resumen y verificación (columnas válidas)
```

```
PRINT '=====';  
  
PRINT 'RESUMEN: Server Audits y Specifications';  
  
PRINT '=====';
```

```
-- mostrar el audit creado
```

```
SELECT audit_id, name AS audit_name, type_desc, on_failure_desc, queue_delay,  
is_state_enabled, create_date, modify_date  
  
FROM sys.server_audits  
  
WHERE name = @AuditName;
```

```
-- detalles de file-audit (si existen)
```

```
SELECT saf.audit_id, sa.name AS audit_name, saf.log_file_path, saf.log_file_name,  
saf.max_file_size, saf.max_rollover_files, saf.reserve_disk_space  
  
FROM sys.server_file_audits saf  
  
INNER JOIN sys.server_audits sa ON saf.audit_id = sa.audit_id  
  
WHERE sa.name = @AuditName;
```

```
-- server audit specs
```

```
SELECT sas.server_specification_id, sas.name AS spec_name, sa.name AS  
audit_name, sas.is_state_enabled, sas.create_date  
  
FROM sys.server_audit_specifications sas  
  
LEFT JOIN sys.server_audits sa ON sas.audit_guid = sa.audit_guid;
```

-- db audit specs (QhatuPeru)

SELECT das.database_specification_id, das.name AS db_spec_name, sa.name AS
audit_name, das.is_state_enabled, das.create_date

FROM QhatuPeru.sys.database_audit_specifications das

LEFT JOIN sys.server_audits sa ON das.audit_guid = sa.audit_guid;

-- 8. Leer eventos si se creó como FILE

IF @CreatedToFile = 1

BEGIN

PRINT 'Intentando leer archivos de auditoría desde: ' + @AuditFilePattern;

BEGIN TRY

SELECT TOP 200 event_time, action_id, succeeded, server_principal_name,
database_name, object_name, statement, host_name, application_name

FROM sys.fn_get_audit_file(@AuditFilePattern, DEFAULT, DEFAULT)

ORDER BY event_time DESC;

END TRY

BEGIN CATCH

PRINT ' ⚠ No se pudieron leer archivos: ' + ERROR_MESSAGE();

PRINT 'Verifica que el servicio de SQL Server tenga permisos sobre la carpeta y
que existan archivos .sqlaudit.';

END CATCH

END

ELSE

BEGIN

PRINT 'El audit fue creado en APPLICATION_LOG. Para ver eventos: Event Viewer -
> Windows Logs -> Application (o Security según configuración).';

END

PRINT 'SCRIPT COMPLETADO.';

3. Justificación técnica de la solución aplicada

- Server Audit es el objeto central que define el destino de los logs (FILE o Event Log) y controla la recolección.
- Registramos los eventos de login (FAILED/SUCCESSFUL) a nivel servidor para detectar accesos no autorizados o patrones sospechosos.
- Creamos una Database Audit Specification en [QhatuPeru](#) que captura grupos de eventos de cambios de esquema y cambios de permisos (SCHEMA_OBJECT_CHANGE_GROUP, DATABASE_OBJECT_PERMISSION_CHANGE_GROUP, etc.), clave para trazabilidad de cambios DDL en objetos críticos.
- La solución es resistente: intenta usar FILE (más práctico) y, si falla por permisos/ruta, utiliza [APPLICATION_LOG](#) para no perder la capacidad de auditoría.
- Se usan especificaciones por grupos de eventos para cubrir categorías comunes y cumplir requisitos regulatorios (SOX/HIPAA/GDPR).

4. Explicación de las buenas prácticas utilizadas

1. Ejecución como SYSADMIN: creación/habilitación de audit requiere privilegios elevados; el script valida al inicio.
2. Fallback controlado: intenta FILE y, si falla, cae a Event Log para garantizar que la auditoría quede habilitada.
3. No eliminar llaves/objetos activos sin control: el script limpia objetos previos de forma segura (OFF → DROP).
4. Mensajes informativos y manejo de errores: TRY/CATCH y mensajes [PRINT](#) para seguimiento y diagnóstico.

5. Uso de columnas válidas en vistas/resultados: las consultas de verificación usan columnas existentes y comprobadas.
6. Rotación y límites: configuración **MAXSIZE**, **MAX_ROLLOVER_FILES** para evitar llenado de disco.
7. Separación Server vs DB: server-audit para eventos de servidor; database-audit-spec para eventos de BD, evitando confusión y permitiendo granularidad.
8. Documentación embebida: encabezados y comentarios que permiten usar el script como documento técnico.

Results

Messages

	audit_id	audit_name	type_desc	on_failure_desc	queue_delay	is_state_enabled	create_date	modify_date
1	65539	Audit_QhatsuPeru_Security	APPLICATION LOG	CONTINUE	1000	1	2025-11-13 15:47:12.150	2025-11-13 15:47:12.150

	audit_id	audit_name	log_file_path	log_file_name	max_file_size	max_rollover_files	reserve_disk_space
--	----------	------------	---------------	---------------	---------------	--------------------	--------------------

	server_specification_id	spec_name	audit_name	is_state_enabled	create_date
1	65539	Spec_Login_QhatsuPeru	Audit_QhatsuPeru_Security	1	2025-11-13 15:47:12.170

	database_specification_id	db_spec_name	audit_name	is_state_enabled	create_date
1	65539	Spec_DDL_QhatsuPeru	Audit_QhatsuPeru_Security	1	2025-11-13 15:47:12.183

Query executed successfully.

SAMIRSSJ (16.0 RTM) | SAMIRSSJ\homer (63) | QhatsuPERU | 00:00:00 | 3 rows

Proyecto 8 — Monitoreo de eventos y alertas con Extended Events + Auditoría

1. Enunciado del ejercicio

Configurar una sesión de Extended Events que capture deadlocks y eventos de login fallidos, guardar la información en un archivo o buffer, y crear una vista en la base QhatsuPeru para consultar los eventos desde la base de datos.

código T-SQL

USE master;

GO

```
PRINT '=====';
```

```
PRINT 'PASO 1: Eliminando sesión existente (si la hay)';
```

```
PRINT '=====';
```

```
IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name =  
'XEvent_QhatuPeru_Monitor')
```

```
    DROP EVENT SESSION XEvent_QhatuPeru_Monitor ON SERVER;
```

```
GO
```

```
PRINT '=====';
```

```
PRINT 'PASO 2: Creando sesión Extended Events';
```

```
PRINT '=====';
```

```
BEGIN TRY
```

```
-- 🌀 IMPORTANTE: Cambia la ruta si tu SQL Server no tiene acceso a C:\XELogs
```

```
DECLARE @AuditPath NVARCHAR(300) = 'C:\XELogs\QhatuPeru_Monitor.xel';
```

```
DECLARE @SQL NVARCHAR(MAX);
```

```
SET @SQL = '
```

```
CREATE EVENT SESSION [XEvent_QhatuPeru_Monitor]
```

```
ON SERVER
```

```
ADD EVENT sqlserver.error_reported
```

```
    (WHERE ([error_number]=(18456))), -- Login failed
```

```
ADD EVENT sqlserver.lock_deadlock
```

```
ADD TARGET package0.event_file (
```

```
    SET filename = N" + @AuditPath + ",
```

```
    max_file_size = 10,
```

```

        max_rollover_files = 5
    )
WITH (
    MAX_MEMORY = 10MB,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY = 5 SECONDS,
    TRACK_CAUSALITY = ON,
    STARTUP_STATE = ON
);';

EXEC (@SQL);

ALTER EVENT SESSION [XEvent_QhatuPeru_Monitor] ON SERVER STATE =
START;

PRINT '✅ Sesión creada y habilitada correctamente.';

END TRY

BEGIN CATCH

    PRINT '❌ ERROR al crear sesión: ' + ERROR_MESSAGE();

END CATCH;

GO

PRINT '=====';

PRINT 'PASO 3: Crear vista para leer eventos desde el archivo';

PRINT '=====';

USE QhatuPeru;

```

GO

IF OBJECT_ID('dbo.Vista_XEvents_QhatuPeru') IS NOT NULL

DROP VIEW dbo.Vista_XEvents_QhatuPeru;

GO

CREATE VIEW dbo.Vista_XEvents_QhatuPeru

AS

SELECT

event_data.value('(event/@name)[1]', 'nvarchar(100)') AS EventName,

event_data.value('(event/@timestamp)[1]', 'datetime2') AS [Timestamp],

event_data.value('(event/data[@name="error_number"]/value)[1]', 'int') AS
ErrorNumber,

event_data.value('(event/data[@name="message"]/value)[1]', 'nvarchar(4000)') AS
Message,

event_data.value('(event/action[@name="client_app_name"]/value)[1]',
'nvarchar(4000)') AS AppName,

event_data.value('(event/action[@name="client_hostname"]/value)[1]',
'nvarchar(4000)') AS HostName,

event_data.value('(event/action[@name="server_principal_name"]/value)[1]',
'nvarchar(4000)') AS LoginName

FROM (

SELECT CAST(event_data AS XML) AS event_data

FROM sys.fn_xe_file_target_read_file('C:\XELogs\QhatuPeru_Monitor*.xel', NULL,
NULL, NULL)

) AS src;

GO

PRINT '  Vista creada correctamente para consultar eventos.';

GO

PRINT '=====';

PRINT 'PASO 4: Validando sesión activa';

PRINT '=====';

SELECT

s.name,

s.startup_state,

CASE

WHEN t.target_name IS NOT NULL THEN '✓ Activa'

ELSE '✗ Inactiva'

END AS Estado,

t.target_name,

t.target_data

FROM sys.server_event_sessions AS s

LEFT JOIN sys.dm_xe_sessions AS x ON s.name = x.name

LEFT JOIN sys.dm_xe_session_targets AS t ON x.address = t.event_session_address

WHERE s.name = 'XEvent_QhatuPeru_Monitor';

GO

3. Justificación técnica de la solución aplicada

- Por qué Extended Events (XE): XE es la plataforma moderna y eficiente de SQL Server para captura de telemetría (reemplaza a SQL Trace). Consume menos CPU y I/O, tiene menor impacto en producción y más flexibilidad en targets y formatos (file, ring_buffer, event_sink, etc.).
- Eventos seleccionados

- `sqlserver.lock_deadlock` (deadlocks): captura el punto donde dos o más transacciones se bloquean mutuamente — información crítica para depurar contención y evitar pérdida de trabajo. Incluye contexto de transacciones y texto de la consulta.
- `sqlserver.error_reported` filtrado por `error_number = 18456` (login failed): 18456 es el error típico de autenticación fallida; registrar estos eventos es esencial para detectar intentos de intrusión o cuentas mal configuradas.
- (Alternativa/Mejora) `sqlserver.failed_login` existe en algunas versiones; si está disponible, puede usarse directamente.
- Target FILE vs RING_BUFFER
 - FILE: recomendado en producción — persistente, permite análisis histórico con `sys.fn_xe_file_target_read_file`, y archivos `.xel` pueden rotarse. Requiere carpeta accesible y permisos de escritura por la cuenta de servicio de SQL Server.
 - RING_BUFFER: fallback en entornos donde no se puede escribir a disco. Mantiene datos en memoria (no persistente tras reinicio) y es útil para diagnósticos rápidos, pero no para auditoría histórica.
- Parámetros de sesión
 - `MAX_MEMORY`: control del buffer interno de XE. Debe establecerse en un valor válido para la instancia (en KB/MB según la versión). Se eligió un valor conservador/seguro (ej.: 10MB) para evitar errores y no impactar memoria general.
 - `MAX_DISPATCH_LATENCY`: balance entre latencia y overhead. 5 segundos ofrece rápida disponibilidad sin altos costes.
 - `EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS`: prioriza estabilidad de la instancia evitando sobrecarga; permite perder algún evento si hay alta presión.
 - `TRACK_CAUSALITY = ON`: facilita correlacionar eventos relacionados (útil para seguir la cadena causal de un deadlock).
- Vista en base de datos
 - `vw_XEvent_QhatuPeru_Monitor` / `Vista_XEvents_QhatuPeru` normaliza la lectura (parseo XML) y permite consultas SQL estándar, integración con

informes o jobs de verificación.

- Uso de `sys.fn_xe_file_target_read_file` para lectura desde .xel y `sys.dm_xe_sessions/sys.dm_xe_session_targets` para ring_buffer.
- Manejo de errores y resiliencia
 - El script intenta crear FILE y hace fallback a ring_buffer si la creación falla por permisos o rutas. Esto asegura que la captura se habilite en la mayoría de escenarios.
 - TRY/CATCH y mensajes informativos facilitan diagnóstico.

4. Buenas prácticas, recomendaciones operativas y consideraciones

1. Ejecución y permisos

- Ejecutar la creación de sesiones como sysadmin.
- La carpeta destino (ej. `C:\XELogs`) debe existir y la cuenta de servicio de SQL Server debe tener permisos de escritura/lectura.

2. Persistencia y retención

- Si usas FILE, configura rotación (`max_file_size`, `max_rollover_files`) y un job que archive/mueva/elimine** archivos antiguos para evitar llenar disco.
- Mantén una política de retención acorde a cumplimiento/regulación (ej.: 90–365 días según políticas).

3. Alertas y respuesta

- Crear un SQL Agent Job (o proceso externo) que:
 - Consulte la vista periódicamente y dispare alertas (email, webhook, SIEM) si se detectan patrones: p. ej. >5 logins fallidos en X minutos por IP/usuario, o N deadlocks en X minutos.
 - Archive eventos críticos en tabla histórica si quieres conservarlos en la BD.

- Integrar con SIEM (Splunk, QRadar, Azure Sentinel) para correlación y detección avanzada.

4. Seguridad

- Limitar quién puede leer archivos XE y quién puede ver la vista: crear rol con permisos mínimos para consultas de auditoría.
- Auditar el acceso a la carpeta de logs (OS-level) y proteger backups.

5. Impacto en rendimiento

- Monitorear CPU y memoria tras habilitar la sesión; ajustar `MAX_MEMORY` y `MAX_DISPATCH_LATENCY` si observas impacto.
- Evitar captura excesiva de datos en producción: elegir eventos y acciones (actions) necesarias y evitar capturar `sql_text` indiscriminadamente en tablas muy activas.

6. Robustez operacional

- Habilitar `STARTUP_STATE = ON` para que la sesión se inicie automáticamente tras reinicio.
- Documentar la ruta de logs, la política de rotación/retención y el procedimiento de restauración/archivo.

7. Pruebas y validación

- Validar en preproducción: generar deadlocks controlados y logins fallidos para confirmar que los eventos llegan a la vista y que el job de alertas responde.
- Probar restauración de escenarios (leer .xel con `sys.fn_xe_file_target_read_file`).

8. Correlación con auditoría (Server Audit)

- Complementar XE con Server/Database Audit cuando necesites trazabilidad oficial (cumplimiento). XE es ideal para diagnosticar y monitorear; Audit para evidencia legal/regulatoria.

Results		Messages			
	name	startup_state	Estado	target_name	target_data
1	XEvent_QhatuPeru_Monitor	1	? Inactiva	NULL	NULL

PROYECTO 9: Automatización de alertas ante eventos críticos (Login Failures y Deadlocks)

1. Enunciado del ejercicio

Implementar un mecanismo automatizado en SQL Server que detecte eventos críticos (como intentos fallidos de inicio de sesión y deadlocks) registrados por Extended Events y genere alertas automáticas mediante un SQL Agent Job.

código T-SQL

```
USE QhatuPERU;
```

```
GO
```

```
PRINT '=====';
```

```
PRINT 'PASO 1: Creando tabla de almacenamiento de eventos críticos';
```

```
PRINT '=====';
```

```
GO
```

```
IF OBJECT_ID('dbo.EventosCriticos', 'U') IS NOT NULL
```

```
    DROP TABLE dbo.EventosCriticos;
```

```
GO
```

```
CREATE TABLE dbo.EventosCriticos (
```

```
ID INT IDENTITY(1,1) PRIMARY KEY,  
TipoEvento NVARCHAR(50),  
FechaHora DATETIME DEFAULT GETDATE(),  
Usuario NVARCHAR(100),  
Host NVARCHAR(100),  
Detalle NVARCHAR(MAX)  
);  
GO
```

```
PRINT '✓ Tabla EventosCriticos creada exitosamente.';  
GO
```

```
PRINT '=====';  
PRINT 'PASO 2: Creando procedimiento para insertar eventos desde XE';  
PRINT '=====';  
GO
```

```
IF OBJECT_ID('dbo.sp_RegistrarEventosXE', 'P') IS NOT NULL  
    DROP PROCEDURE dbo.sp_RegistrarEventosXE;  
GO
```

```
CREATE PROCEDURE dbo.sp_RegistrarEventosXE  
AS  
BEGIN  
    SET NOCOUNT ON;
```

```

DECLARE @Now DATETIME = GETDATE();

INSERT INTO dbo.EventosCriticos (TipoEvento, Usuario, Host, Detalle)

SELECT

    event_data.value('(event/@name)[1]', 'nvarchar(50)'),

    event_data.value('(event/action[@name="server_principal_name"]/value)[1]',
'nvarchar(100)'),

    event_data.value('(event/action[@name="client_hostname"]/value)[1]',
'nvarchar(100)'),

    event_data.value('(event/data[@name="message"]/value)[1]', 'nvarchar(max)')

FROM (

    SELECT CAST(event_data AS XML) AS event_data

    FROM sys.fn_xe_file_target_read_file('C:\XELogs\QhatuPERU_*.xel', NULL,
NULL, NULL)

) AS X

    WHERE event_data.value('(event/@name)[1]', 'nvarchar(50)') IN ('error_reported',
'lock_deadlock');

PRINT '✓ Eventos críticos registrados correctamente en tabla.';

END;

GO

PRINT '=====';

PRINT 'PASO 3: Creando SQL Agent Job para ejecutar cada 5 minutos';

PRINT '=====';

GO

-- Crear el Job (esto se ejecuta en msdb)

```

```
USE msdb;
```

```
GO
```

```
IF EXISTS (SELECT 1 FROM msdb.dbo.sysjobs WHERE name =  
'Job_Alerta_EventosCriticos')
```

```
EXEC msdb.dbo.sp_delete_job @job_name = 'Job_Alerta_EventosCriticos';
```

```
GO
```

```
EXEC msdb.dbo.sp_add_job
```

```
@job_name = 'Job_Alerta_EventosCriticos',
```

```
@enabled = 1,
```

```
@description = 'Monitorea XE y registra eventos críticos (deadlocks y logins fallidos).';
```

```
GO
```

```
EXEC msdb.dbo.sp_add_jobstep
```

```
@job_name = 'Job_Alerta_EventosCriticos',
```

```
@step_name = 'RegistrarEventos',
```

```
@subsystem = 'TSQL',
```

```
@database_name = 'QhatuPERU',
```

```
@command = 'EXEC dbo.sp_RegistrarEventosXE;';
```

```
GO
```

```
EXEC msdb.dbo.sp_add_schedule
```

```
@schedule_name = 'Cada5Minutos',
```

```
@freq_type = 4,
```

```
@freq_interval = 1,
```

```
@freq_subday_type = 4,
```



```
@freq_subday_interval = 5,  
@active_start_time = 0000;  
GO
```

```
EXEC msdb.dbo.sp_attach_schedule  
@job_name = 'Job_Alerta_EventosCriticos',  
@schedule_name = 'Cada5Minutos';  
GO
```

```
EXEC msdb.dbo.sp_add_jobserver  
@job_name = 'Job_Alerta_EventosCriticos';  
GO
```

```
PRINT '✓ Job de alerta automática creado y programado exitosamente.';  
GO
```